



*Asociación Profesional del Cuerpo Superior
de Sistemas y Tecnologías de la Información
de la Administración del Estado*

Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.

TEMAS ESPECÍFICOS II: Tecnología básica

**Tema 77. Lenguajes y herramientas para la utilización de redes
globales. HTML, CSS y XML. Navegadores web y
compatibilidad con estándares**

Actualización 2017: Javier Rincón Abel

Actualización 2015: Alberto García Sola

Actualización 2017

ÍNDICE

1	HTML (HYPERTEXT MARKUP LANGUAGE).....	4
1.1	INTRODUCCIÓN A LA WEB.....	4
1.1.1	HTTP (HYPERTEXT TRANSFER PROTOCOL)	4
1.1.2	URL (UNIFIED RESOURCE LOCATORS).....	5
1.2	EVOLUCIÓN HISTÓRICA DE HTML.....	6
1.3	SINTAXIS DE HTML.....	6
1.4	DOCUMENT OBJECT MODEL (DOM).....	8
1.5	HTML5.....	9
1.6	CSS (CASCADING STYLE SHEETS).....	10
1.6.1	CSS3.....	13
1.6.2	PREPROCESADORES CSS	15
1.6.3	RESPONSIVE WEB DESIGN / ADAPTATIVE WEB DESIGN.....	15
1.7	JAVASCRIPT	16
1.7.1	TECNOLOGÍAS RELACIONADAS CON JAVASCRIPT	17
1.7.1.1	AJAX.....	17
1.7.1.2	JSON.....	17
1.8	GENERACIÓN DINÁMICA DE HTML.....	17
1.8.1	GENERACIÓN DINÁMICA EN EL CLIENTE	18
1.8.2	GENERACIÓN DINÁMICA EN EL SERVIDOR.....	19
1.8.2.1	CGI (COMMON GATEWAY INTERFACE)	19
1.8.2.2	JAVA SERVLETS.....	19
1.8.2.3	JSP (JAVA SERVER PAGES).....	19
1.8.2.4	ASP (ACTIVE SERVER PAGES) Y ASP.NET.....	19
1.8.2.5	PHP (HYPERTEXT PROCESSOR).....	20
1.8.2.6	OTROS.....	20
1.9	WEB 2.0, WEB 3.0 Y WEB SEMÁNTICA	20
1.10	LA WEB EN LA ADMINISTRACIÓN PÚBLICA	21
2	XML (EXTENSIBLE MARKUP LANGUAGE)	22
2.1	INTRODUCCIÓN A XML	22
2.2	COMPARACIÓN XML – HTML.....	22
2.3	INTEROPERABILIDAD ENTRE APLICACIONES Y XML. SERVICIOS WEB XML (SOAP, WSDL Y UDDI).....	23
2.3.1	SERVICIO WEB, PROVEEDOR DEL SERVICIO Y SOLICITANTE DEL SERVICIO	25
2.3.2	SOAP (SIMPLE OBJECT ACCESS PROTOCOL)	25
2.3.3	WSDL (WEB SERVICES DESCRIPTION LANGUAGE).....	27
2.3.4	UDDI (UNIVERSAL DESCRIPTION DISCOVERY AND INTEGRATION)	29
2.4	SINTAXIS DE XML	29
2.5	DEFINICIÓN DE LENGUAJES Y VALIDACIÓN DE DOCUMENTOS XML (DTDs Y XML SCHEMAS).....	30
2.5.1	DTDs (DOCUMENT TYPE DEFINITIONS)	30
2.5.2	XML SCHEMAS	31

2.6	PROCESADO DE DOCUMENTOS XML (DOM Y SAX)	34
2.7	TRANSFORMACIÓN Y PRESENTACIÓN DE DOCUMENTOS XML (XSL Y XSLT) ..	37
2.8	LENGUAJES BASADOS EN XML	39
2.9	SINDICACIÓN Y XML.....	40
2.9.1	RSS Y RDF	40
2.9.2	ATOM	41
2.9.3	ESTADO ACTUAL	42
2.10	XML Y LA INFORMACIÓN FINANCIERA: XBRL	42
2.10.1	XBRL EN ESPAÑA	43
2.11	DESVENTAJAS DE XML Y ALTERNATIVAS	44
2.12	NORMATIVA APLICABLE	44
3	NAVEGADORES WEB Y COMPATIBILIDAD CON ESTÁNDARES.....	46
3.1	NAVEGADORES WEB.....	46
3.1.1	HISTORIA	46
3.1.2	CONCEPTOS	46
3.1.3	FUNCIONAMIENTO DE UN NAVEGADOR	47
3.1.4	COMPONENTES DE UN NAVEGADOR.....	48
3.1.4.1	INTERFAZ GRÁFICA DE USUARIO	49
3.1.4.2	MOTOR DE RENDERIZADO	49
3.1.4.3	MOTORES JAVASCRIPT	50
3.1.4.4	COMPLEMENTOS	51
3.1.5	NAVEGACIÓN SEGURA, PRIVADA Y ANÓNIMA	51
3.1.5.1	NAVEGACIÓN SEGURA	51
3.1.5.2	NAVEGACIÓN PRIVADA.....	51
3.1.5.3	NAVEGACIÓN ANÓNIMA: TOR, I2P Y PROXIES.....	52
3.1.6	PRINCIPALES NAVEGADORES	53
3.1.7	COMPARATIVA DE CUOTA DE MERCADO DE NAVEGADORES.....	54
3.2	COMPATIBILIDAD CON ESTÁNDARES	56
3.2.1	ESTÁNDARES WEB.....	56
3.2.2	WEB STANDARDS PROJECT (WASP)	57
3.2.3	CUMPLIMIENTO DE ESTÁNDARES	58
3.2.4	EL CROSS-BROWSING	61
4	CONCLUSIONES	63

1 HTML (HyperText Markup Language)

1.1 Introducción a la web

La Web, entendida como servicio o aplicación de Internet, sigue la arquitectura cliente – servidor. Es decir, para que la Web pueda funcionar necesita de (al menos):

- Un servidor web.
- Un cliente web, que también se denomina navegador web o browser.

Esto se puede apreciar en la Figura 1:

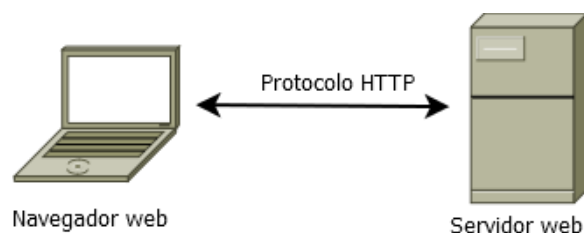


Figura 1. Arquitectura cliente/servidor

El servidor web es la aplicación responsable de esperar conexiones de los navegadores web, aceptar y procesar sus peticiones, y devolver documentos (por ejemplo, una página web HTML) mediante respuestas.

Por su parte, el navegador web es la aplicación responsable de establecer conexiones con los servidores web, solicitar documentos mediante peticiones, y procesar y presentar los documentos recibidos en las respuestas.

El protocolo de comunicaciones mediante el que dialogan los navegadores y servidores web es HTTP (Hypertext Transfer Protocol) que, junto con HTML, es la otra piedra angular de la Web.

Dado que existe una gran diversidad de documentos, es necesario establecer un mecanismo que permita identificar unívocamente cada documento y localizarlo. Esto se consigue por medio de URLs (Uniform Resource Locator). Una URL hace referencia al identificador único de cada recurso disponible en Internet.

Por su importancia dentro de la web, las siguientes secciones describen brevemente HTTP y URL.

1.1.1 HTTP (HyperText Transfer Protocol)

Como se ha adelantado anteriormente, HTTP (HyperText Transfer Protocol) es un protocolo de la capa de aplicación que se utiliza para transferir recursos¹ de la Web. Aunque el estándar HTTP no lo especifica, lo más habitual es su uso junto con TCP (a nivel de transporte) e IP (a nivel de red). Es un protocolo estandarizado por IETF (Engineering Task Force) y su última versión es HTTP/2.

HTTP se basa en la arquitectura cliente/servidor. El cliente HTTP abre una conexión y envía un mensaje de petición (request message) al servidor. Entonces, el servidor HTTP envía un mensaje de respuesta (response message) que contiene el recurso solicitado por el cliente. Tras el mensaje de respuesta el servidor cierra la conexión, es por esto que HTTP es un protocolo sin estado (*stateless*), ya que no mantiene el estado entre las distintas transacción de un mismo cliente. Para mantener dicho estado entre transacciones es frecuente el uso de cookies (pequeña información que el servidor puede almacenar en el cliente para consultar actividad previa del usuario).

¹ Se entiende recurso por cualquier trozo de información que puede ser identificado mediante una URL (fichero HTML, imagen, sonido, etc.)

Tanto el mensaje de petición como de respuesta siguen el mismo formato: línea inicial, cero o más encabezados, una línea en blanco y opcionalmente, el cuerpo del mensaje (en la respuesta será el recurso solicitado). La línea inicial para una petición consistirá:

- Comando que indica la acción que desea que se efectúe sobre el recurso identificado: GET, PUT, POST, OPTIONS, TRACE, DELETE,...
- El identificador del recurso (URI).
- La versión del protocolo HTTP en uso.

Para una respuesta será:

- Versión de HTTP.
- Código de estado (p.e 200 OK, 404 Not Found, etc.).
- Frase explicativa.

En 1992 se creó, basado en HTTP, el protocolo HTTPS (Hypertext Transfer Protocol Secure) para ofrecer un medio más apropiado de transmisión para información sensible en el protocolo HTTP. HTTPS utiliza cifrado basado en SSL/TLS para crear un canal cifrado entre el servidor y el cliente.

La versión HTTP/1.1 surgió en 1999 para abordar nuevas necesidades y solucionar algunos problemas de HTTP/1.0. La principal mejora de esta versión es que se permite que en una conexión se realicen varias transacciones solicitud/respuesta, lo que puede reducir el tiempo de respuesta considerablemente.

Otras mejoras incluyen el ahorro de ancho de banda a través del uso de caché, la posibilidad de incluir servidores virtuales basados en nombres para un uso eficiente de direcciones IP y la posibilidad de permitir que una respuesta se envíe aun cuando no se conoce su longitud completa (chunked response).

En mayo de 2015 se publicó el RFC 7540 con la especificación de HTTP/2, la cual pretende ofrecer mayor velocidad, reducir el uso de recursos en la red y el servidor, mejorar el rendimiento de la red y comprensión de cabeceras.

Una de las características más importantes de esta versión es que es un protocolo binario en lugar de texto, permitiendo más seguridad y menos errores. Además, para la compresión de cabeceras se ha empleado el algoritmo HPACK, más seguro que GZIP (utilizado en SPDY).

HTTP/2 está basado en SPDY ("SPeeDY"), un protocolo complementario a HTTP presentado por Google con el objetivo de incrementar la velocidad de navegación. Cuando se comenzó a trabajar en HTTP/2 Google anunció que dejaría de soportar SPDY para apoyar la nueva versión HTTP.

1.1.2 URL (Unified Resource Locators)

Como ya introducíamos, la Web consiste en una enorme colección de documentos, dispersos por servidores de todo el mundo, e interconectados mediante enlaces. Por tanto, se vuelve necesario disponer de un esquema de direccionamiento que permita ubicar y solicitar esos documentos.

Este esquema de direccionamiento es el que proporcionan las URLs (Unified Resource Locators). Las URLs no son más que una forma estándar de especificar la ubicación de los documentos en la Web.

La sintaxis básica de una URL responde al siguiente esquema:

`http://host:port/path`

donde host es el servidor donde reside el documento, port es el puerto TCP donde escucha el servidor, y path su ruta relativa dentro del servidor.

No obstante, las URLs también admiten una sintaxis más general dando lugar al concepto global de URI (Uniform Resource Identifier) al incluir la información después de "path", como sigue:

`scheme://host:port/path/extra-path-info?query-info`

donde host, port y path tienen el significado ya visto, scheme especifica el protocolo de comunicaciones (HTTP o HTTPS, FTP, etc.) y extra-path-info?query-info es información adicional

para la generación dinámica de HTML, y que como se ha mencionado al añadirse a la URL conforma la URI.

Para conocer todos los detalles de las URLs se puede consultar el RFC 1738 (<http://www.faqs.org/rfcs/rfc1738.html>).

1.2 Evolución histórica de HTML

HTML (HyperText Markup Language) es un lenguaje de marcado para la elaboración de páginas web estandarizado por el W3C. HTML permite describir el contenido y formato de los documentos de la Web de una forma muy sencilla, lo que permite que incluso usuarios no técnicos sean capaces de utilizarlo.

La primera versión formal de HTML surgió en 1991 a partir de un trabajo iniciado por el físico Tim Berners-Lee en 1980. Sin embargo, la primera versión estandarizada de HTML (por el IETF) fue la segunda versión de HTML, HTML 2.0, en 1995. Desde 1996, los estándares HTML los publica el W3C.

En 1997 se publicó la versión HTML 3.2, la cual introducía algunas mejoras, como el uso de applets de Java. Un año más tarde se publicó la HTML 4.0, el cual supuso un gran avance con respecto a las versiones anteriores. Se introdujo el uso de hojas de estilo (CSS), tablas complejas, mejoras en la accesibilidad y formularios, etc.

Con la revisión y actualización de HTML en HTML 4.01 en 1999 se paralizó el desarrollo de HTML y el W3C se centró en el desarrollo del estándar XHTML (del que se hablará más adelante).

XHTML (eXtensible Hypertext Markup Language) es una variante de HTML más rigurosa y compleja. Consiste en expresar las páginas HTML mediante XML. La W3C pretendía que XHTML fuese el futuro, sin embargo, no era compatible con la mayoría de contenido web existente y su uso era mucho más complejo, por lo que perdía el atractivo inherente a HTML para usuarios no técnicos.

Dado el poco éxito que tuvo XHTML, en el año 2004, Apple, Mozilla y Opera anunciaron su interés en seguir trabajando en HTML y organizaron la asociación WHATWG (Web Hypertext Application Technology Working Group), que centró su trabajo en el estándar HTML5.

Debido a la fuerza que adquirió la organización, en 2007 el W3C decidió unirse al proyecto de la WHATWG y en 2009 se abandonó XHTML para regresar a HTML. Finalmente, en 2014 se publicó la versión final de HTML5, la cual introduce numerosas ventajas y cambios que los desarrolladores llevaban años esperando. En este tema se describirá sintaxis básica de HTML y, por su importancia, las novedades introducidas en esta última versión.

1.3 Sintaxis de HTML

HTML es un lenguaje de etiquetas (también llamado lenguaje de marcas) que permite describir el contenido y formato de los documentos web. El documento web más básico se denomina página web o página HTML. Una página web es un documento, descrito en lenguaje HTML, y conformado normalmente por texto (es decir, información para el usuario), instrucciones de formateo (colores, tamaños, tipos de letra, etc.), enlaces para navegar a otras páginas web (por eso se habla de hipertexto), imágenes, sonidos, animaciones, etc.

Interesa resaltar aquí que las páginas web contienen tanto información para el usuario (ej. un texto) como instrucciones de formateo (ej. colores, tamaños, tipos de letra, etc.). Esta será una de las diferencias clave con XML que, al contrario que HTML, separará completamente información o contenido de formateo.

Las páginas HTML se dividen en dos partes: la cabecera (<head></head>) y el cuerpo (<body></body>). La cabecera contiene la información de la propia página, por ejemplo título o idioma. A excepción del título, esta información no se visualiza. Por otra parte, el cuerpo de la página incluye el contenido de la página que el usuario ve, tanto texto, imágenes, etc., como información para su formateo.

De esta forma, una página web básica tendrá la siguiente apariencia:

```
<html>

  <head>

    <title>Página de ejemplo</title>

  </head>

  <body>

    <p>
      Ejemplo <b>básico</b> de página HTML.
      Es importante notar la mezcla de información y formateo.
      <!-- Esto es un comentario en HTML -->
    </p>

  </body>

</html>
```

Como se puede apreciar se mezcla tanto información de la página (título y contenido) cómo instrucciones para el formateo (por ejemplo `<b>` para texto en negrita o `<p>` para indicar un párrafo).

Algunas de las etiquetas más utilizadas son para:

- Formateo de texto: negrita `<b>`, cursiva `<i>`, subrayado `<u>`, títulos (`<h1>`, `<h2>`, etc.), párrafos `<p>`, salto de línea `<br />`, fuente (`<font face="arial" size=8 color=red>Texto</ font >`), etc.
- Listas: `<ul>` para listas no ordenadas y `<ol>` para ordenadas.

```
<UL>

  <LI> Primer item

  <LI> Segundo item

  <LI> Tercer item

</UL>
```

- Tablas:

```
<table>

  <tr>

    <td>Celda 1</td>

    <td>Celda 2</td>

    <td>Celda 3</td>

  </tr>

  <tr>

    <td>Celda 4</td>

    <td>Celda 5</td>

    <td>Celda 6</td>

  </tr>

</table>
```

- Imágenes: ``

- Enlaces: `<a href="http://www.google.es">Link a Google</a>`
- Formularios: la etiqueta `<form>` nos permite definir formularios. Contiene un atributo `action` obligatorio para indicar el documento que procesará los datos del formulario:

```
<form action="datosPersonales.php" method="post">
  Nombre: <input type="text" name="nombre"> </br>
  Apellido: <input type="text" name="apellido"> </br>

  Sexo:
  <input type="radio" name="sexo" value="hombre">Hombre
  <input type="radio" name="sexo" value="mujer">Mujer
  </br>

  Aficiones:
  <input type="checkbox" name="afic" value="musica">Música
  <input type="checkbox" name="afic" value="cine">Cine
  </br>

  <input type="submit" value="Enviar">
  <input type="reset" value="Borrar">
  </br>
</form>
```

La apariencia que tendría el formulario al mostrarse en un navegador se muestra en la Figura 2:

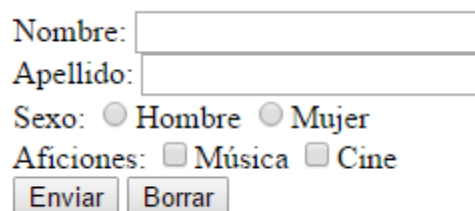


Figura 2. Formulario HTML

La sintaxis de HTML es extensa, ya que dispone de muchas palabras reservadas o etiquetas. No es el propósito del presente tema hacer una revisión exhaustiva de la sintaxis de HTML y sus etiquetas. Para ello, se recomienda al lector revisar las referencias especializadas (<http://www.w3.org/TR/html/semantics.html>).

Como se ha dicho, la sintaxis de HTML mezcla tanto contenido como formato, lo cual dificulta el mantenimiento y desarrollo de las páginas web. Con el objetivo de separar contenido – formato, existen varias propuestas, entre las más exitosas está las hojas de estilo CSS (Cascading Style Sheets) y XML, que serán vistas en los siguientes apartados.

1.4 Document Object Model (DOM)

Para permitir que los lenguajes de programación pudieran extraer información o manipular cualquier elemento de una página web o de un XML era necesario definir de alguna manera qué tipos de

elementos conforman una página web, cómo nombrarlos y cómo se relacionan entre sí. Con este objetivo el W3C definió un estándar denominado DOM (Document Object Model).

DOM es una API que proporciona un conjunto estándar de objetos para representar documentos HTML y XML y un modelo estándar sobre cómo pueden combinarse dichos objetos. El DOM representa los objetos de un documento HTML o XML organizados jerárquicamente. De esta forma, los programas pueden acceder y modificar el contenido, estructura y estilo de los documentos HTML y XML.

Las diferentes versiones del DOM son conocidas como DOM Nivel 0, DOM Nivel 1, DOM Nivel 2, DOM Nivel 3 y actualmente la última especificación publicada es DOM Nivel 4.

Dado que se usa tanto para XML como para HTML, se volverá a retomar este tema en la sección correspondiente de XML.

1.5 HTML5

Tal como se ha comentado en la introducción, la última revisión de HTML, HTML5, introduce importantes novedades que los desarrolladores llevaban reclamando mucho tiempo. Actualmente, el término HTML5 es comúnmente usado para referirse al conjunto HTML5+CSS3+Javascript, como se verá en apartados posteriores.

Las novedades más importantes introducidas en HTML5 implican:

- Es adaptable, es decir, permite realizar diseños que se adaptan a diferentes dispositivos (web, tablets, móviles...). De hecho, ofrece una mayor compatibilidad con los navegadores de distintos dispositivos.
- Simplifica el código provocando páginas más ligeras y, por tanto, más rápidas, lo que favorece tanto la usabilidad como la indexación en buscadores.
- Incorpora nuevas capacidades JavaScript con motores más potentes. Por ejemplo, es posible ejecutar múltiples JavaScript en paralelo en una misma página y ejecutar páginas sin estar conectado. De hecho, desde Microsoft se ha fomentado el desarrollo de aplicaciones de escritorio para Windows 8 utilizando HTML5.
- Mayor soporte para la web semántica. Incorpora nuevas etiquetas y metadatos que favorecen el posicionamiento SEO y la accesibilidad.
- Mejora el soporte a contenido multimedia posibilitando la inserción de vídeos y audio directamente. Ofreciendo, además, soporte a codecs específicos.
- Reduce la necesidad del uso de plugins, como flash y otros scripts.
- Mejora en formularios.
- Nuevas APIs: geolocalización, drag & drop, etc.

En concreto se han eliminado etiquetas, algunos atributos de etiquetas y se han modificado y añadido otras para dar un mayor soporte a la web semántica, mejorar los formularios, añadir nuevas funcionalidades, etc. A continuación se enumeran algunas de las etiquetas más importantes que han sido introducidas o eliminadas. Puede consultar toda la sintaxis en <http://www.w3.org/TR/html5/>.

Elementos borrados:

- Referentes a presentación: <basefont>, , <center>, <big>, <u>, <strike>. Deben definirse mediante CSS.
- Atributos de formato de algunas etiquetas: atributo background de <body>, atributo align y border, cellpadding y cellspacing de las tablas. Deben definirse mediante CSS.
- Elementos que habían caído en desuso por ser reemplazables: por ejemplo <dir> (se usa) o <applet> (se usa <embed> o <object>).
- Etiquetas problemáticas como <frame> (reemplazada por <iframe>, <frameset>, <noframes>)

Nuevos elementos:

- Web semántica: se propone la sustitución de <div> para maquetar mediante el uso de <header>, <footer>, <nav> (zona de navegación), <section>, <article>, <aside> (para añadir detalles, aclaraciones,...). Aunque puede seguir utilizando <div> para maquetación. En la Figura 3 se muestra un ejemplo de maquetación utilizando estas etiquetas.

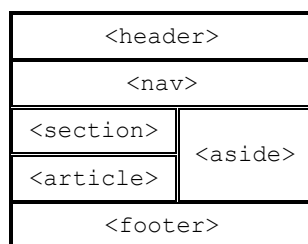


Figura 3. Maquetación HTML5

- Nuevas necesidades como <audio>, <video> y la mejora de <canvas> (capaz de renderizar elementos 3D), <svg> (gráficos vectoriales).
- Mejoras en los formularios: fecha, color, email, rango... Permiten al navegador saber exactamente qué información espera.
- Etiquetas para manejar grandes conjuntos de datos: Datagrid, Details, Menu y Command. Enfocados a la generación de tablas dinámicas, que permiten filtrar, ordenar y ocultar contenido en cliente.

Las APIs más interesantes introducidas son:

- HTML Geolocation: usada para obtener la información geográfica del usuario.
- HTML Drag and Drop: permite “agarrar” un objeto y arrastrarlo a otra localización. Dado que esta API es parte del estándar, cualquier elemento puede ser “draggable”.
- HTML Local Storage: permite que las aplicaciones web puedan almacenar datos de usuario dentro del navegador. Algo que hasta ahora se hacía mediante el uso de cookies con sus consiguientes problemas (seguridad, rendimiento).
- HTML Application Cache: permite que una aplicación web sea cacheada, y por tanto, puede ser accedida posteriormente offline, se mejora la velocidad y se reduce la carga del servidor.
- HTML Web Workers: un web worker es un JavaScript que se ejecuta en background, de forma independiente a otros script. Esto permite que no se afecte el rendimiento de la página y se pueda continuar navegando sin bloqueos mientras se ejecuta el web worker.
- HTML SSE (Server-Sent Events): permite actualizar una página web automáticamente sin necesidad de tener que preguntar al servidor si hay algún cambio disponible, como ocurría antes.

La introducción de estas mejoras y el conjunto HTML5+CSS3+Javascript ha experimentado un gran auge en los últimos años propiciando un incremento masivo del uso de Internet en teléfonos móviles, tablets y otros dispositivos.

1.6 CSS (Cascading Style Sheets)

Como se describía anteriormente, HTML mezcla información para el usuario (ej. un texto) con instrucciones de formato (ej. colores, tamaños, tipos de letra, etc.). Sin embargo, para facilitar el mantenimiento de las páginas web, sería deseable que información y formato estuvieran separados.

Un primer intento de lograr esta separación información – formato fueron las hojas CSS (Cascading Style Sheets), aunque ha habido otros intentos posteriores más exitosos (ej. XML). Las hojas CSS son un estándar del W3C.

Las hojas CSS permiten que las páginas web tengan dos partes claramente diferenciadas:

- Una parte donde se definen los estilos o formatos (<style> ... </style>)
- Y otra parte de contenido o información (<body> ... </body>) donde se aplican los estilos.

Esto se puede apreciar en la siguiente página web de ejemplo, donde se define un estilo para h1:

```
<html>

  <head>

    <title>Ejemplo de Document-level style</title>
    <style type="text/css">
      h1 {color: red; font-family: Arial; font-size: large;}
    </style>
  </head>

  <body>
    <h1>Texto con el estilo definido arriba</h1>
  </body>
</html>
```

En realidad, las hojas CSS permiten ir un paso más allá en la separación entre información y formato. Así, la definición de los estilos, que en el ejemplo anterior se hace dentro de la propia página HTML (Document-level Styles), se puede extraer a otros ficheros separados (External Styles). De este modo, además, se fomenta la reutilización de los estilos entre varias páginas web de una aplicación o sitio.

Esto se puede observar en el siguiente ejemplo, donde los estilos se han definido en el fichero externo /css/mis_estilos.css:

```
<html>

  <head>

    <title> Estilos enlazados </title>
    <link rel="stylesheet" type="text/css"
href="css/mis_estilos.css" media="screen" />
    <style> <!-- OTROS ESTILOS --> </style>
  </head>

  <body>
    <h1>Texto con el estilo definido en el fichero externo</h1>
  </body>
</html>
```

Donde el fichero CSS tendrá la siguiente forma:

```
h1 {color: red; font-family: Arial; font-size: large;}
```

Finalmente, las hojas CSS permiten una tercera y última forma de definir estilos (Inline Styles), aunque es la menos recomendada porque es la que menor separación información – formato consigue. Sólo se recomienda en aquellos casos en que se desee modificar el formato de un elemento concreto muy específico y que no se va a volver a utilizar. En esta modalidad, la definición y aplicación del estilo se dan conjuntamente en el contexto de una etiqueta HTML; por ejemplo:

```
<h1 style="color: blue; font-style: italic"> Texto con el estilo  
definido en la propia etiqueta </h1>
```

En definitiva, las hojas CSS permiten definir estilos de tres formas diferentes que, de menor a mayor separación información – formato son:

- Inline Styles.
- Document-level Styles.
- External Styles.

Es posible consultar todos sus detalles en <http://www.w3.org/Style/CSS/>.

Es importante resaltar que antes de que existiera CSS, los desarrolladores utilizaban tablas (<table>) para maquetar las web (es decir, estructurar espacialmente el contenido). Esta solución presentaba diversos problemas, ya que generaba un código complejo y difícil de reutilizar, no se adapta a pantallas pequeñas y generaba, por lo general, webs más pesadas e ineficientes. Desde la aparición de CSS, la maquetación con hojas de estilo simplifica y mejora mucho este proceso. Aquí podemos ver un ejemplo de una web maquetada con CSS:

```
<html>  
  <head>  
    <title> Estilos enlazados </title>  
    <link rel="stylesheet" type="text/css" href="estilo.css"  
media="screen" />  
  </head>  
  <body>  
    <div id="contenedor">  
      <div id="cabecera"> Texto cabecera</div>  
  
      <div id="pie"> Texto pie </div>  
    </div>  
  </body>  
</html>
```

```
#contenedor{
width:80%;
background-color: #f5f5f5;
margin: 0 auto 0 auto;
font-family:Verdana;
}
#cabecera{
width:100%;
background-color: green;
text-align: center;
overflow:hidden;
padding: 0px 0px 0px 0px;
margin: 0 auto 10px auto;
}
#cuerpo{
width: 100%;
margin: 0px auto 0px auto;
background-color: #f5f5f5;
overflow: hidden;
}
#pie{
background-color: yellow;
margin: 10px 0 0 0;
padding: 0 0 0 0px;
text-align:center;
clear: both;
font: 8pt Verdana;
}
```

Como resultado de la aplicaciones de los estilos CSS sobre el código HTML podemos ver como se mostraría en la Figura 4.

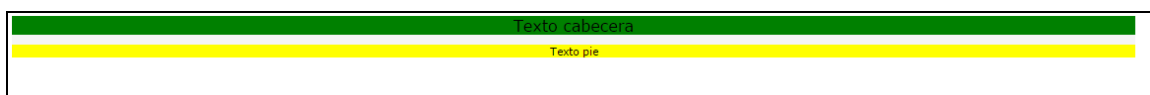


Figura 4. Página web maquetada con CSS

1.6.1 CSS3

CSS se ha creado en diferentes niveles (especificaciones), donde cada nivel se construye sobre el anterior, dotándolo generalmente de mayor funcionalidad y preservando la compatibilidad hacia atrás.

Actualmente la última especificación es CSS3 (desde 2011). A diferencia de los anteriores niveles, CSS3 está estructurado en módulos, por lo tanto, cada módulo se encuentra en un estado diferente de estabilidad y desarrollo.

Algunas de las novedades incluyen la mejora en el tratamiento de los fondos y bordes (CSS3 Backgrounds and Borders Module). Por ejemplo, se ha incluido la posibilidad de establecer múltiples imágenes de fondo ('background-clip', 'background-break', 'background-origin', etc.), crear sobras (box-shadow), poner los bordes redondeados (border radius), incluir transparencia en cualquier elemento (opacity), etc.

Otras novedades están relacionadas con la apariencia del texto, como por ejemplo, la posibilidad de partir las palabras con guiones al final de línea o de instalar la fuente de la letra que se desee sin que el usuario deba tenerla instalada en su ordenador. Con CSS3 es posible también dividir el texto en columnas utilizando las propiedades column-count, column-gap y column-rule.

Otra importante característica incluida en CSS3 es el nuevo modelo de cajas flexibles (propiedad Flexible Box o flexbox), el cual permite establecer los elementos de una página para que se comporten de forma predecible en diferentes tamaños de pantalla y dispositivos. Esta característica todavía se encuentra en estado "W3C Candidate Recommendation" (candidata a recomendación).

Por último, una de las grandes novedades de CSS3 es la posibilidad de crear animaciones de forma sencilla y sin utilizar plugins adicionales. Esto se realiza mediante la propiedad animation y sus subpropiedades, que nos permiten determinar el tiempo y duración de la animación (animation-delay, animation-name, animation-duration, animation-iteration-count, animation-play-state, etc.).

Para especificar la apariencia utilizaremos reglas @keyframes, que describe los fotogramas de la animación. La subpropiedad animation-name indica la regla @keyframes que describe los fotogramas de la animación. El fotograma usa percentage para indicar en qué momento de la secuencia de la animación tiene lugar: 0% (from) es el inicio y 100% (to) es el final. También se pueden incluir fotogramas para pasos intermedios. Haciendo uso de javascript, es posible usar eventos (objeto AnimationEvent) para un mayor control de las animaciones. A continuación se muestra un ejemplo de animación que cambia el color del elemento de forma gradual.

```
div {
  -webkit-animation: myfirst 5s; /* Chrome, Safari, Opera */
  animation: myfirst 5s;
}

/* Chrome, Safari, Opera, -webkit para compatibilidad con
navegadores antiguos */
@-webkit-keyframes myfirst {
  0%   {background: red;}
  25%  {background: yellow;}
  50%  {background: blue;}
  100% {background: green;}
}

/* Standard syntax */
@keyframes myfirst {
  0%   {background: red;}
  25%  {background: yellow;}
  50%  {background: blue;}
  100% {background: green;}
}
```

También es posible rotar los elementos en 2D y 3D mediante la propiedad transform o crear efectos cuando se cambia de un estilo a otro usando la propiedad transition.

Dado que, como se ha comentado en este tema, CSS3 está estructurado en módulos, cada módulo se encuentra en un nivel de estabilidad y desarrollo. Esto está dando lugar a que algunos módulos se encuentren ya en un nivel 4 de desarrollo (por ejemplo, el módulo Selectors). Al conjunto de estos módulos se le conoce como CSS4, pero no será hasta que todos los módulos del nivel 3 se encuentren completamente desarrollados cuando podamos hablar realmente de CSS4.

1.6.2 Preprocesadores CSS

En los últimos años han ganado mucha popularidad los preprocesadores de CSS. Los preprocesadores CSS permiten operaciones avanzadas que no ofrece CSS directamente. Los preprocesadores utilizan la sintaxis del preprocesador específico y lo convierten al código CSS final que será interpretado por el navegador. Así, por ejemplo, permiten el uso de variables, reutilización de bloques de código, usar funciones, estructuras de control, etc.

Actualmente existen muchos preprocesadores CSS. Algunos de los más utilizados son Sass² (utiliza Ruby), Less (escrito en Javascript) y Stylus³ (utiliza Node.js). Los tres son preprocesadores bastante potentes, sin embargo LESS⁴ es el más extendido debido a que es más fácil de utilizar (aunque ligeramente menos potente), es usado en Twitter Bootstrap y cuenta con una comunidad de usuarios muy activa (al igual que Sass). Existen numerosos frameworks que pueden ayudarnos en el uso de estos preprocesadores, como Twitter Bootstrap, Compass o Jeet Framework.

1.6.3 Responsive web design / Adaptative web design

Con el creciente uso de diferentes dispositivos para acceder a la web (tablets, móviles, etc.) se hace imprescindible una forma de diseñar las webs de tal forma que se vean adecuadamente en diferentes dispositivos y resoluciones. Debido a esto el responsive web design (o diseño web adaptable) ha tenido una gran aceptación en los últimos años.

Responsive web design es un conjunto de técnicas que permite que la página web se adapte al medio a través del cual se está accediendo y visualizando. Responsive web design se apoya en tres conceptos fundamentales:

1. Uso de Media Queries (incluidas en CSS3). Permiten aplicar estilos condicionalmente teniendo en cuenta los parámetros de la pantalla.
2. Diseño fluido. En lugar de diseñar la web basándonos en valores fijos (por ejemplo width: 960px), el diseño fluido está pensado en términos de proporciones. Así se definen los layouts en base a porcentajes que se ajustan a la pantalla proporcionalmente.
3. Elementos fluidos dentro de los layouts, como por ejemplo, las fuentes, imágenes o elementos multimedia también serán adaptados proporcionalmente.

Con el uso de responsive design sólo es necesaria una versión de nuestro HTML y CSS para que se visualice correctamente en cualquier dispositivo y resolución.

Relacionado con el responsive design tenemos otra aproximación llamada adaptative design. Ambos enfoque buscan el mismo objetivo que consiste en adaptar el interfaz a las dimensiones del dispositivo para facilitar la interacción con el usuario. La diferencia entre ambos está en que adaptative design define unos rangos de resolución de pantallas (viewports) en los que el comportamiento es el mismo y el cambio se produce solo al saltar de rango de resolución de pantalla. En cambio el responsive design los cambios son continuos al cambiar la resolución, es decir los cambios son más fluidos, suaves y continuos.

Las dos aproximaciones son igual de validas pero hay algunas características asociadas a cada enfoque que conviene conocer. El responsive design requiere más trabajo de diseño y de implementación, pero se suele considerar la carga más rápida. En cambio el adaptative design se considera menos flexible para adaptarse a cualquier dispositivo, pero más sencillo y rápido de implementar.

² <http://sass-lang.com/>

³ <http://learnboost.github.io/stylus/>

⁴ <http://lesscss.org/>

1.7 JavaScript

JavaScript es un lenguaje de programación muy relacionado con HTML y, de hecho, permite interactuar con el Modelo de Objetos del Documento (DOM). Los programas de JavaScript se incrustan en las páginas HTML, o se vinculan a ellas, y se ejecutan en el navegador web con el objeto de dotar a las páginas web de mayor interactividad con el usuario.

Un ejemplo típico de programa JavaScript es aquel que, vinculado a un formulario HTML, permite validar la sintaxis de los campos contra unos patrones (ej. un campo de fecha contra el patrón "dd/mm/aaaa"), antes de que los datos del formulario sean enviados al servidor web.

Las características básicas de JavaScript son:

- Es un lenguaje de scripting (interpretado, no compilado). Los programas de JavaScript se denominan scripts.
- Es un lenguaje orientado a objetos. Si bien suele considerarse un lenguaje orientado a objetos, existe cierta controversia sobre si debería ser considerado orientado a objetos en el sentido estricto.
- Los scripts se incrustan o se vinculan a las páginas HTML (mediante las etiquetas HTML `<script> ... </script>`).
- Los scripts se descargan, junto con las páginas HTML, desde el servidor hasta el navegador web.
- Los scripts se ejecutan en el navegador web, no en el servidor.

A continuación se muestra un ejemplo sencillo de script JavaScript que, a partir de un nombre proporcionado por el usuario, lo saluda mediante una caja de diálogo:

```
<html>
  <head>
    <script language="Javascript">
      function dameNombre(nombre) {
        alert("Hola " + nombre + "!");
      }
    </script>
  </head>
  <body>
    Por favor, escriba su nombre:
    <form>
      <input type="text" name="name" onBlur="dameNombre
(this.value)" value="">
    </form>
  </body>
</html>
```

Aunque la versión inicial de JavaScript fue desarrollada por Netscape, posteriormente surgieron diversas versiones del mismo lenguaje, casi siempre con pequeñas incompatibilidades entre sí. Así, existe la versión JScript, que es la versión de JavaScript desarrollada por Microsoft. También existe ECMAScript, que es la versión de JavaScript estandarizada por la ECMA (European Computer Manufacturers Association).

No es el objetivo de este tema profundizar en JavaScript ni en sus tecnologías relacionadas (sobre esto se profundiza en el tema 114). Pero dada su gran importancia en el mundo web, se dará una breve descripción de estas.

1.7.1 Tecnologías relacionadas con JavaScript

1.7.1.1 AJAX

JavaScript es uno de los lenguajes más populares para desarrollo web. Sin embargo, presenta limitaciones respecto a su potencia para desarrollar páginas dinámicas que interactúen con el usuario. Estas limitaciones junto con la aparición de Flash disminuyeron su popularidad durante unos años. La llegada de AJAX devolvió de nuevo la fama a JavaScript.

El término AJAX es un acrónimo de Asynchronous JavaScript + XML. En realidad, AJAX no es una tecnología en sí misma, si no la agrupación de varias tecnologías:

- XHTML o HTML y hojas de estilos en cascada (CSS) para la presentación.
- DOM para interactuar dinámicamente con la información presentada.
- El objeto XMLHttpRequest para intercambiar datos de forma asíncrona con el servidor web.
- XML y XSLT para el intercambio y manipulación de datos.
- JavaScript

AJAX permite una comunicación asíncrona con el servidor web gracias al motor AJAX escrito en JavaScript. Gracias a esto es posible realizar cambios sobre una página web sin necesidad de recargarla ni de que el usuario realice ninguna acción, lo que aumenta en gran medida la interactividad, velocidad y usabilidad de las páginas. Es precisamente esta característica la que propició el auge de AJAX en el desarrollo web.

1.7.1.2 JSON

JSON (JavaScript Object Notation) es un formato para el intercambio de información. El formato JSON permite representar estructuras de datos y objetos en forma de texto. En JavaScript un texto representado en JSON es muy sencillo de analizar, por esto se ha generalizado su uso frente a XML, especialmente en AJAX.

A continuación se muestra un pequeño ejemplo de JSON sacado de <http://json.org/example>.

```
{
  "id": 69,
  "name": "HTML and XML",
  "price": 55.00,
  "tags": ["html", "xml", "ASTIC"]
  "url": "http://www.astic.es/opositor/adquirir-el-temario"
}
```

1.8 Generación dinámica de HTML

Mediante HTML sólo es posible crear sitios Web estáticos, lo cual implica que el usuario no puede interactuar con él. En la actualidad, y dado el gran desarrollo de la web, esto no es suficiente. La

combinación de HTML con otras tecnologías permite la generación dinámica de páginas web. A esta combinación suele referirse bajo el nombre de DHTML (Dynamic HTML). De esta forma, cuando el servidor Web recibe una petición para una determinada página, la página se genera automáticamente por el software como respuesta directa a la petición de la página. Esto es conocido como página web dinámica.

La generación del contenido dinámico puede hacerse en el servidor o en el cliente, empleándose por lo general lenguajes distintos en cada caso, aunque algunos lenguajes pueden trabajar en ambos lados.

En la generación dinámica en el servidor, cuando el servidor recibe una petición, ejecuta una aplicación determinada y envía el resultado al cliente. El navegador cliente interpreta la respuesta como un HTML normal y lo visualiza. Por otro lado, cuando la generación es en lado del cliente, al recibir la petición el servidor, este envía una respuesta que contiene código que el cliente es capaz de entender y ejecutar para realizar una determinada acción y después visualizarla.

La generación en el lado del cliente tiene la ventaja de que se evita sobrecargar al servidor, sin embargo, su limitación estriba en que el servidor sólo podrá enviarle código que sea capaz de entender. Además, la ejecución en el lado del servidor puede realizarse sin tener en cuenta al cliente y un ambiente controlado.

Como se ha dicho, normalmente se utilizan lenguajes y tecnologías diferentes en el lado cliente y servidor. En el lado cliente podemos encontrar, por ejemplo, controles ActiveX, objetos embebidos tipo Flash, applets, AJAX, etc.

En el lado del servidor es posible utilizar lenguajes embebidos en código HTML (como PHP, ASP o JSP), enlaces a ejecutables (tipo CGI o SSI), objetos (Servlets) e incluso lenguajes que separan la presentación de la lógica de negocio (ASP.Net de Microsoft).

Aunque no es el objetivo de este tema (existen temas específicos para algunas de las tecnologías aquí nombradas) profundizar en cada una de estas tecnologías, se dará un breve repaso sobre las características principales de algunas de las más importantes.

1.8.1 Generación dinámica en el cliente

De entre las tecnologías para generación dinámica en el lado del cliente destaca especialmente Javascript junto con AJAX, de las que ya se ha hablado anteriormente en este tema. Sin embargo existen otras tecnologías que también se tratan en profundidad en el tema 62.

Cabe resaltar también en este apartado, aunque actualmente su uso es cada vez menor, la utilización de plugins específicos como Flash o Silverlight. Este tipo de tecnología requiere la instalación de plugins específicos en el navegador para poder visualizarse.

Entre ellos, el que ha gozado de más popularidad es Adobe Flash (hasta 2005, Macromedia Flash). Adobe Flash es el nombre bajo el que se hace referencia tanto al formato, como al programa de edición multimedia, como al reproductor de SWF (Shockwave Flash) llamado Adobe Flash Player.

El formato Flash utiliza gráficos vectoriales e imágenes ráster, sonido, código de programa, flujo de vídeo y audio bidireccional (el flujo de subida sólo está disponible si se usa conjuntamente con Flash Media Server). En sentido estricto, Flash es el entorno y Flash Player es el programa de máquina virtual utilizado para ejecutar los archivos generados con Flash, de forma que los archivos de Flash, generalmente con extensión de archivo SWF, pueden aparecer en una página web para ser vista en un navegador, o pueden ser reproducidos independientemente por un reproductor Flash.

Flash fue creado inicialmente para servir como soporte de animaciones sencillas, de tipo GIF animado o los archiconocidos banners, pero el apoyo que encontró entre los webmasters y los diseñadores web fue asombroso, apoyado en su facilidad de uso, la calidad de los resultados, la rápida descarga y la profesionalidad de los resultados, entre otros.

Sin embargo, en los últimos años el uso de Flash ha decaído sustancialmente debido, en parte, al gran auge de los dispositivos móviles, la mayoría de los cuales no tienen la capacidad de cómputo suficiente como para ejecutar una animación Flash. Junto a esto, Flash ha presentado también

muchas vulnerabilidades de seguridad. Todo esto ha provocado que actualmente su uso no sea tan común como lo era hace unos años.

1.8.2 Generación dinámica en el servidor

Como se ha comentado existen numerosas tecnologías para la generación dinámica en el servidor. Aquí se expondrán algunas de las más importantes.

1.8.2.1 CGI (Common Gateway Interface)

CGI (Common Gateway Interface) es una de las primeras y más básicas tecnologías para la generación web dinámica. Especifica un estándar, definido en el RFC 3875, que determina el mecanismo de comunicación entre el servidor web y una aplicación externa en cualquier lenguaje, de tal forma que desde dicha aplicación puedan recibirse los datos enviados por el cliente y el resultado le sea devuelto para visualizarse.

En CGI el cliente indica la aplicación a ejecutar y los argumentos mediante una URL que forma la petición HTTP del cliente. El servidor transfiere dicha información a la aplicación para que esta la ejecute y forme una respuesta, normalmente como documento HTML. La respuesta es pasada al servidor y redirigida por este al cliente para su visualización.

De esta forma, la aplicación puede estar en cualquier lenguaje. Sin embargo, el problema de CGI es el rendimiento ante múltiples peticiones, ya que para cada una debe crearse un nuevo proceso en el servidor. Existen algunas propuestas, como extensiones o módulos del servidor para cada lenguaje (p.e., `mod_perl`, ISAPI de Microsoft o NSAPI de Netscape) o FastCGI, que intentan resolver este problema. Otras alternativas a CGI son Servlets o ASP.

1.8.2.2 Java Servlets

Un servlet es un componente web desarrollado en Java que permite extender la funcionalidad del servidor. Un servlet es invocado a través de una URL y pueda generar contenido dinámico como texto, imágenes y sonido, aunque normalmente será contenido HTML, que es devuelto al cliente para visualizarlo.

Al ser Java un entorno multihilo, los servlets están diseñados para permitir dar respuesta a un alto número de solicitudes concurrentes, lo que soluciona el problema visto con CGIs.

1.8.2.3 JSP (Java Server Pages)

JSP es otra de las tecnologías, junto con Servlets, para generación dinámica de páginas web usando Java. Las páginas JSP combinan código HTML con código Java, mezclando por tanto, componentes estáticos y dinámicos.

Para desplegar y correr JavaServer Pages, se requiere un servidor web compatible con contenedores servlets como Apache Tomcat o Jetty. En realidad, JSP está basado en servlets, de hecho, el motor JSP realiza la traducción de una página JPS a servlet la primera vez que se invoca.

1.8.2.4 ASP (Active Server Pages) y ASP.NET

ASP es la tecnología para la creación de páginas dinámicas del lado del servidor desarrollada por Microsoft. En realidad ASP no es un lenguaje de programación, si no que ofrece un marco sobre el que construir las páginas, que podrán ser desarrolladas utilizando lenguajes de script, como por ejemplo Visual Basic Script, JScript o PerlScript. Este código de script será embebido dentro de las páginas HTML.

La filosofía de ASP es similar a la vista en JSP, sin embargo, ASP sólo funciona en Microsoft Windows, ya que requiere el servidor IIS (Internet Information Server). También es posible ejecutarlo sobre servidores Unix/Linux utilizando el framework Mono.

La evolución de ASP es ASP.NET. Las páginas de ASP.Net, conocidas como Web Forms, están construidas sobre el Common Language Runtime (CLR), lo que permite utilizar cualquier lenguaje soportado por la plataforma .NET (VB.NET, C#, JScript.Net).

Se puede profundizar en ambas tecnologías en el tema 115.

1.8.2.5 PHP (Hypertext Processor)

PHP es uno de los lenguajes más usados en la actualidad. El código PHP es embebido dentro del código HTML e interpretado por el servidor mediante un módulo de procesador de PHP que genera la página resultado. Su gran aceptación en el mundo web es debida a que es un lenguaje muy flexible, potente, de código abierto y soportado por la mayoría de servidores web y sistemas operativos y es código abierto.

Estas características son las que han permitido que PHP sea elegido como tecnología de servidor por múltiples sitios web con gran demanda de tráfico, como por ejemplo Facebook. De hecho, la combinación de Linux+Apache+MySQL+PHP, más conocida como LAMP, es la más utilizada para definir la infraestructura de un servidor, existiendo incluso paquetes para su instalación y configuración conjunta.

1.8.2.6 Otros

Existen otros muchos lenguajes en el lado del servidor como Python, Ruby, Perl, etc. Últimamente, una tendencia que está tomando cada vez más relevancia es Javascript del lado del servidor. Existen varias alternativas para esto: Node.js, Jaxer, EJS, RingoJS, y AppengineJS. De entre ellas la más utilizada hoy día es Node.js⁵. Node.js es una plataforma para el desarrollo de aplicaciones escalables construida sobre el intérprete de JavaScript de Google Chrome y puede ejecutarse en Windows, Linux o Mac OS X. Está enfocado en maximizar el rendimiento y la eficiencia mediante un sistema de comunicación asíncrono y orientado a eventos.

1.9 Web 2.0, Web 3.0 y Web Semántica

El término Web 2.0 hace referencia a la evolución que ha experimentado la web. En constante progresión, ha pasado de unas primeras páginas estáticas en HTML (Web 1.0), a un segundo nivel más elaborado (Web 1.5), caracterizado por la creación “al vuelo” de documentos dinámicos. Pero los cambios que se intuyen ahora son más profundos y complejos. Este nuevo estadio de la Web es el que se conoce como Web 2.0.

Aunque no existe una definición clara sobre qué es la Web 2.0, se considera que la Web 2.0 comprende aquellos sitios web que facilitan el compartir información, la interoperabilidad, el diseño centrado en el usuario y la colaboración. Ejemplos de Webs 2.0 sería las redes sociales, wikis, blogs, etc. En ocasiones se ha relacionado el término Web 2.0 con el de Web semántica. Sin embargo ambos conceptos, corresponden más bien a estados evolutivos de la web, y la Web semántica correspondería en realidad a una evolución posterior, a la Web 3.0 o web inteligente.

La Web Semántica se basa en la idea de añadir metadatos semánticos y ontologías a la web para que las máquinas “entiendan” a las personas y procesen de una forma eficiente la avalancha de información publicada en la Web. Por tanto se podría identificar la Web semántica como una forma de Web 3.0. Así, el término acuñado como Web 3.0 englobaría la web semántica, la web de la nube, la web de las aplicaciones y la web multidispositivo.

El propio HTML5 comentado en este tema incluye etiquetas semánticas y permite una mayor descripción de los datos, así como el soporte junto con JavaScript y CSS3 para la denominada Web 2.0. Sin embargo cuando hablamos de ontologías se utilizan preferiblemente otros lenguajes tales

⁵ <https://nodejs.org/>

como RDF (basado en XML) y OWL (basado en RDF), en los que no se profundizará, pues el tema 120 de este temario ya profundiza ampliamente en los conceptos de Web 2.0, Web 3.0 y Web semántica.

1.10 La Web en la Administración Pública

La Norma Técnica de Interoperabilidad de Catálogo de estándares establece un catálogo formado por un conjunto mínimo de estándares que satisfacen lo previsto en el artículo 11 del Real Decreto 4/2010, de 8 de enero, y que dan soporte al resto de Normas Técnicas de Interoperabilidad.

Las tecnologías incluidas en este catálogo relacionadas con HTML, incluyendo sus versiones mínimas aceptadas son:

- HTML, 4.01
- CSS, 2.1
- MHTML, RFC 2557
- XHTML, 1.0
- HTTP, 1.1
- URI, RFC 3986 y RFC 5785
- URL RFC 1738
- Uniform Resource Names (URN) Namespaces

Se hace necesario también referenciar a la importancia que se da a la accesibilidad web desde la Administración Pública: “se dice que una página o sitio web es accesible cuando está diseñado y construido para que sus contenidos y servicios estén disponibles para cualquier persona, con independencia de sus capacidades visuales, auditivas, cognitivas o motrices e independientemente de la tecnología que utilizan.”

Existen varias leyes que establecen como derechos de los ciudadanos la accesibilidad de diferentes servicios públicos, siendo el más preciso el Real Decreto 1494/2007, de 12 de noviembre, por el que se aprueba el Reglamento sobre las condiciones básicas para el acceso de las personas con discapacidad a la sociedad de la información. Para una información detallada sobre accesibilidad y usabilidad el lector debería referirse al tema 42 de este temario, en el que se profundiza en estos conceptos haciendo especial hincapié a su punto de vista desde la Administración.

2 XML (eXtensible Markup Language)

2.1 Introducción a XML

XML (eXtensible Markup Language) es un lenguaje de marcado⁶ que permite describir información o contenido de forma completamente separada de su presentación o formato. XML está estandarizado por el W3C (World Wide Web Consortium), siendo su versión actual XML 1.1.

En realidad, XML es más que un lenguaje; es un metalenguaje. Se dice que XML es un metalenguaje porque permite definir otros lenguajes o gramáticas. Así, usando XML se puede definir un lenguaje adecuado para, por ejemplo, el intercambio de datos entre editores de libros. O se puede definir otro lenguaje adecuado para describir páginas web (éste es el caso de XHTML). Y así una infinidad de lenguajes para aplicaciones concretas.

XML es el resultado de la simplificación y adaptación a Internet de SGML (Standard Generalized Markup Language), otro lenguaje de marcado previo a XML y más complejo que éste.

En la Figura 5 podemos observar un timeline de los estándares del W3C que une las tecnologías del mundo XML y el mundo de la Web Semántica. A lo largo de las siguientes secciones se describirán algunas de estas tecnologías, habiéndose hablado ya de la Web Semántica.

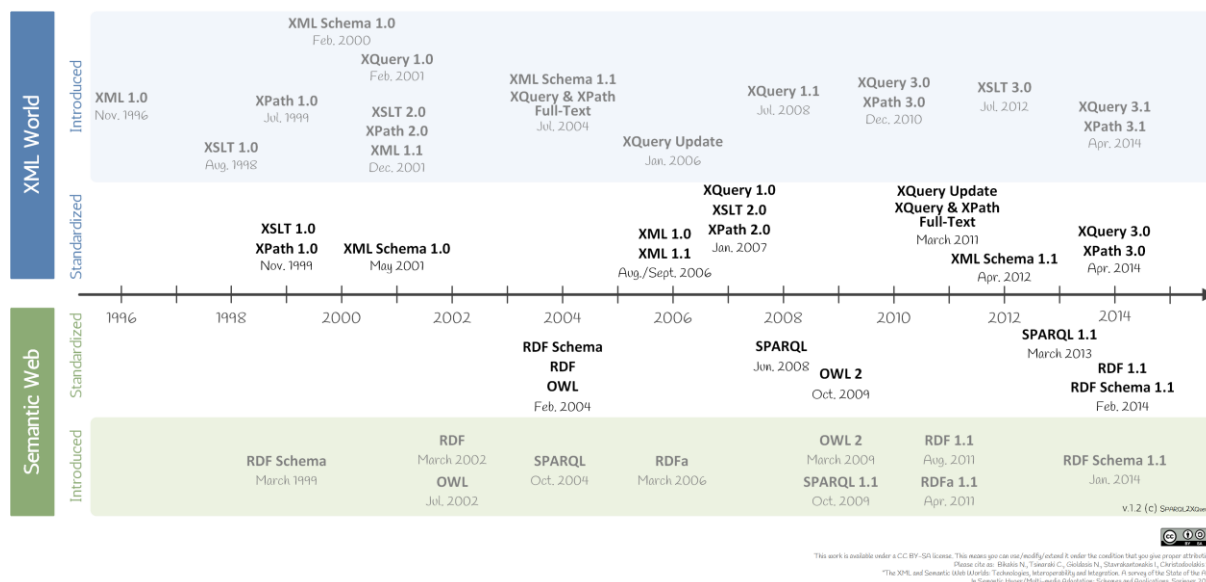


Figura 5. Evolución histórica de XML y la Web Semántica

2.2 Comparación XML – HTML

XML y HTML son lenguajes similares pero con diferencias muy importantes. Por ello, para comprender XML resulta útil compararlo con HTML.

Como ya se adelantó en el apartado dedicado a HTML, HTML también es un lenguaje de marcado. Las etiquetas de HTML son fijas, es decir, si se consulta el estándar de HTML se verá que sus etiquetas son <html>, <head>, <body>, <title>, etc. Son éstas y no otras.

Además, los documentos HTML se caracterizan por mezclar información y presentación, es decir, en un mismo documento o fichero HTML aparecen mezclados los datos (ej. un texto) y las instrucciones

⁶ Lenguaje de marcado es lo mismo que lenguaje de etiquetas. Los lenguajes de marcado / etiquetas tienen etiquetas como <html>, <head>, <body>, <title>, etc.

sobre cómo presentarlos (ej. color, tamaño de letra, etc.). Además, los documentos HTML están pensados para ser presentados en un navegador web, y no para intercambiar datos entre aplicaciones informáticas.

Finalmente, la sintaxis de HTML es un tanto caótica, ya que dependiendo de la versión de HTML hay etiquetas que requieren cierre (ej. `<html> ... </html>`) y otras que no lo requieren (ej. `<p>`), los atributos pueden usar comillas o apóstrofes (" o '), las mayúsculas y minúsculas se consideran indistinguibles, etc. Esto, a su vez, lleva a que la interpretación que los navegadores hacen del HTML no sea uniforme. Así, es habitual que navegadores distintos (ej. Internet Explorer y Mozilla) presenten una misma página HTML de formas ligeramente distintas, dando lugar a tener que realizar modificaciones AdHoc y utilizar funcionalidades específicas para cada navegador con el fin de conseguir un resultado lo más uniforme posible.

Por su parte, XML es otro lenguaje de marcado. Sin embargo, las etiquetas y atributos de éstas en XML son ilimitadas. No se puede decir que las etiquetas de XML sean `<A>`, `<B>`, `<C>`, etc., porque esto no es así. Lo que tiene XML es una serie de mecanismos que se verán más adelante (DTDs y XML Schemas) y que permiten definir otros lenguajes de marcado para aplicaciones concretas. Serán estos otros lenguajes de marcado basados en XML los que tendrán etiquetas fijas. Así, un lenguaje definido para intercambiar datos entre editores de libros tendrá etiquetas como: `<libro>`, `<título>`, `<autor>`, `<precio>`, etc.

Adicionalmente, los documentos XML se caracterizan por separar completamente información y presentación, es decir, en un documento o fichero XML sólo aparece información⁷ (ej. los datos de un libro) y su metainformación, y nunca las instrucciones sobre cómo presentarlos (ej. color, tamaño de letra, etc.). Por eso, los documentos XML están pensados para intercambiar datos entre aplicaciones informáticas.

Finalmente, la sintaxis de XML es formal y precisa. Al contrario de lo que ocurre con HTML, no caben interpretaciones distintas de un mismo documento XML.

2.3 Interoperabilidad entre aplicaciones y XML. Servicios Web XML (SOAP, WSDL y UDDI)

Como se ha adelantado en el apartado anterior, XML está especialmente pensado para intercambiar datos entre aplicaciones informáticas. Por este motivo, muchas veces se dice que "XML facilita la interoperabilidad de las aplicaciones". Esto se puede observar en la Figura 6:

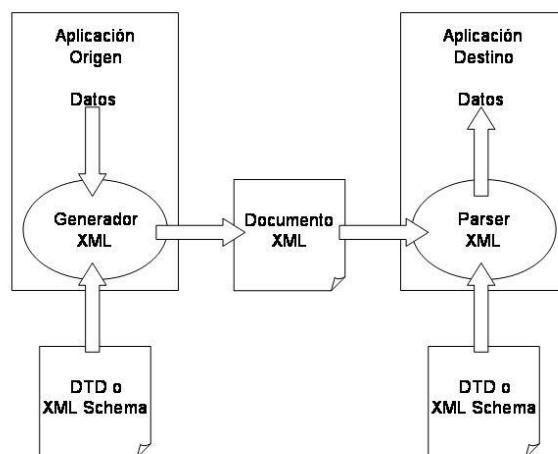


Figura 6. Generación y procesamiento de documento XML

⁷ Esta información aparece estructurada. Es decir, no hay datos sin más. Los datos están organizados, ordenados, estructurados.

Esto es así, precisamente, porque XML permite definir lenguajes de marcado que facilitan el intercambio de datos entre aplicaciones, independientemente de cuál sea su naturaleza (lenguaje de programación y sistema operativo). A la hora de diseñar un proceso de intercambio de datos como el anterior, se pueden adoptar dos enfoques extremos:

- Diseñar un proceso ad-hoc para las aplicaciones que se quiere comunicar. Con este enfoque el esquema del documento XML (DTD o XML Schema) sería algo específico, y el protocolo de comunicaciones también.
- Diseñar un proceso estándar que sirva tanto para estas aplicaciones como para cualesquiera otras que se quieran comunicar mediante XML. Con este enfoque el esquema del documento XML (DTD o XML Schema) sería estándar (como por ejemplo en SOAP), y el protocolo de comunicaciones también (HTTP).

Adoptar un enfoque estándar es especialmente útil en el caso de integrar aplicaciones heterogéneas (ej. están programadas en lenguajes de programación diferentes, se ejecutan sobre plataformas diferentes, etc.) y distribuidas (dispersas geográficamente), porque todos los interesados en una materia (ej. todos los editores de libros) pueden recurrir a esos lenguajes estándar basados en XML para el intercambio de datos entre ellos.

Por eso últimamente han proliferado tantísimo los estándares basados en XML. Hoy en día hay estándares XML para casi cualquier aplicación que se pueda imaginar (ej. WML, XSL, RSS, etc.). Algunos “estándares” llegan a imponerse, y otros no tanto. Pues bien, éste es precisamente el caso de los Servicios Web XML, donde han aparecido múltiples estándares ampliamente extendidos.

Por Servicios Web XML se entiende un conjunto de estándares, entre los que destacan SOAP, WSDL y UDDI, que sirven para comunicar de forma estándar aplicaciones heterogéneas y distribuidas.

En realidad, ha habido muchos intentos previos de conseguir el mismo objetivo (comunicar aplicaciones heterogéneas y distribuidas), siendo algunos de los más notables:

- RPCs (Remote Procedure Calls).
- CORBA (Common Object Request Broker Architecture).
- DCOM (Distributed Component Object Model).
- RMI (Remote Method Invocation).
- Etc.

Entonces, ¿cuál es la principal aportación o novedad de los Servicios Web XML que está haciendo que tengan tanto éxito y superen a sus antecesores? Su principal aportación es que todos sus estándares giran en torno a XML y otros protocolos sencillos de Internet como HTTP. Así, cuando dos aplicaciones se comunican mediante Servicios Web XML:

- El formato de los mensajes que intercambian es XML (y más en particular SOAP).
- El protocolo de comunicaciones que utilizan para intercambiar esos mensajes es HTTP⁸.

El uso de XML y HTTP hace que los Servicios Web XML dispongan de una serie de ventajas sobre sus antecesores, entre las que destacan las siguientes:

- Interoperabilidad: Todos los lenguajes de programación y sistemas operativos soportan XML y HTTP.
- Ubicuidad: Al ser HTTP el protocolo de comunicaciones, los servicios web están disponibles en toda la Internet, y no solamente en redes internas (intranets) como ocurría con sus antecesores.
- Bajo acoplamiento: Las aplicaciones que se comunican mediante servicios web están poco acopladas, ya que pueden estar implementadas en lenguajes de programación distintos, ejecutarse sobre plataformas distintas, etc.
- Reutilización: Una misma funcionalidad implementada como servicio web puede ser reutilizada en muchas aplicaciones, fomentando así la reutilización del software.

⁸ Existe la posibilidad de utilizar otros protocolos como FTP, SMTP, etc., pero lo más habitual es que se utilice HTTP.

- Amplio soporte: Hoy por hoy, son muchos los fabricantes de software que soportan el desarrollo de servicios web mediante librerías, entornos de desarrollo, entornos de ejecución, documentación, etc.
- Rápido aprendizaje: En general, el aprendizaje de XML y HTTP es mucho más sencillo que el aprendizaje de arquitecturas complejas como CORBA y los productos comerciales que la implementan (Orbix, etc.).

En definitiva, mediante el uso de Servicios Web XML se fomenta la integración de aplicaciones mediante el uso de estándares, evitando así las integraciones ad-hoc que tanto se han empleado en este campo. Los Servicios Web XML son un estándar del W3C (<http://www.w3.org/2002/ws/>).

A continuación se revisan los conceptos fundamentales de los Servicios Web XML.

2.3.1 Servicio web, proveedor del servicio y solicitante del servicio

Un servicio web es algo similar a una función en un lenguaje de programación convencional. Se trata de una funcionalidad o algoritmo que se pone a la disposición de otras aplicaciones remotas para que sea invocada por éstas.

Un servicio web tiene dos componentes fundamentales:

- Interfaz: La interfaz de un servicio web es su descripción externa (nombre del servicio, parámetros de entrada, parámetros de salida, dirección de transporte, etc.). Las aplicaciones remotas que utilicen el servicio web sólo necesitan conocer su interfaz; pueden abstraerse de sus detalles de implementación.
- Implementación: La implementación de un servicio web es su codificación en un lenguaje de programación concreto. La misma interfaz puede estar implementada en lenguajes de programación distintos (ej. Java, C++, C#, etc.).

Esta distinción entre interfaz e implementación, que es lo que permite que las aplicaciones a integrar tengan menor acoplamiento, puede apreciarse en la Figura 7:

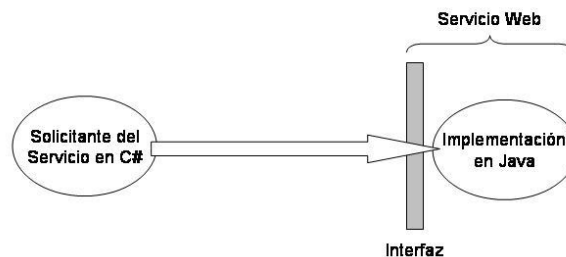


Figura 7. Funcionamiento Servicio Web

La aplicación que ofrece y exporta el servicio web se llama proveedor del servicio. La aplicación que utiliza el servicio web se llama solicitante del servicio.

2.3.2 SOAP (Simple Object Access Protocol)

SOAP es un estándar del W3C (<http://www.w3.org/TR/soap/>). Su versión actual es SOAP 1.2.

Al contrario de lo que pudiera parecer por su nombre (SOAP = Simple Object Access Protocol), SOAP no es un protocolo de comunicaciones, como lo son IP, TCP o HTTP. SOAP es, en realidad, una gramática XML que describe el formato de los documentos XML a intercambiar entre el solicitante y el proveedor del servicio (peticiones y respuestas).

Más en particular, se suele decir que SOAP define un sobre (un formato de mensaje) en el que “ensobrar” las peticiones del solicitante al proveedor y las respuestas del proveedor al solicitante. Por tanto, SOAP sirve para implementar RPCs (llamadas remotas a procedimientos) entre el solicitante y el proveedor del servicio.

SOAP puede funcionar sobre cualquier protocolo de transporte (HTTP, FTP, SMTP, etc.). Sin embargo, lo más habitual es que los mensajes SOAP se transporten sobre HTTP. Esto se puede apreciar en la Figura 8:

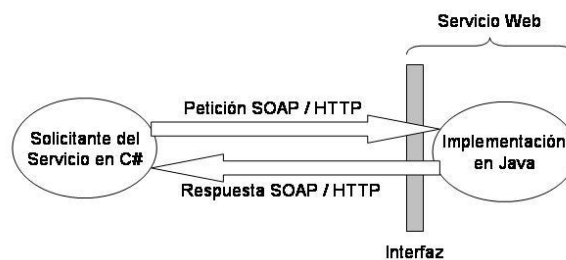


Figura 8. SOAP sobre HTTP

El programador de SOAP no tendrá que generar los mensajes SOAP ni las peticiones / respuestas HTTP manualmente. Antes al contrario, el programador utilizará una librería SOAP, lo que le permitirá pasar fácilmente entre objetos del lenguaje de programación (ej. Java) y mensajes SOAP. Una de las librerías SOAP más utilizadas es Axis para Java.

La apariencia que tiene una petición SOAP es como sigue (nótese que es un documento XML):

```
<SOAP-ENV:Envelope
  xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <SOAP-ENV:Body>
    <ns1:dameCambio xmlns:n1="urn:demo1:exchange"
      SOAP-
ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding">
      <pais1 xsi:type="xsd:string">USA</pais1>
      <pais2 xsi:type="xsd:string">Francia</pais2>
    </ns1:dameCambio>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Por otro lado, la apariencia que tiene una **respuesta SOAP** es como sigue (de nuevo, un documento XML):

```
<SOAP-ENV:Envelope
  xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">

  <SOAP-ENV:Body>
    <ns1:dameCambioRespuesta xmlns:n1="urn:demo1:exchange"
      SOAP-
ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding">

      <return xsi:type="xsd:float">145.42</return>
    </ns1:dameCambioRespuesta>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Para conocer todos los detalles sobre SOAP se pueden consultar las **referencias especializadas** (<http://www.w3.org/TR/soap/>).

2.3.3 WSDL (Web Services Description Language)

En el apartado anterior se ha presentado cómo los servicios web disponen de interfaz e implementación.

La interfaz es el conjunto de detalles (nombre del servicio, parámetros de entrada, parámetros de salida, dirección de transporte, etc.) que deben conocer los solicitantes del servicio para poder acceder al mismo. Por otro lado, la implementación es la codificación del servicio web en un lenguaje de programación concreto y, por tanto, es irrelevante para los solicitantes del servicio.

Pues bien, existe una gramática XML que permite describir la interfaz de un servicio web. Esta gramática XML es WSDL (Web Services Description Language). WSDL es un estándar del W3C, siendo su última versión WSDL 2.0.

Las descripciones WSDL, que en última instancia son documentos XML, son elaboradas por el proveedor del servicio para describir su servicio web, y son compartidas con los solicitantes del servicio para que éstos sepan cómo y dónde acceder a aquél⁹. Una forma cómoda de compartir las descripciones WSDL es mediante directorios UDDI (ver apartado siguiente), aunque existen otras posibilidades como simplemente intercambiar los documentos XML correspondientes.

⁹ Para los conocedores de CORBA, la descripción WSDL de un servicio web es el equivalente a la descripción IDL (Interface Definition Language) de un objeto remoto. Tanto WDSL como IDL describen la interfaz del servicio web u objeto remoto, es decir, el conjunto de operaciones que soportan.

A continuación se puede ver un ejemplo de descripción WSDL (obsérvese que es un documento XML):

```
<?xml version="1.0"?>
<definitions name="StockQuote"

  targetNamespace="http://example.com/stockquote.wsdl"
  xmlns:tns="http://example.com/stockquote.wsdl"
  xmlns:xsd1="http://example.com/stockquote.xsd"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns="http://schemas.xmlsoap.org/wsdl/">

  <types>
    <schema targetNamespace="http://example.com/stockquote.xsd"
      xmlns="http://www.w3.org/2000/10/XMLSchema">
      <element name="TradePriceRequest">
        <complexType>
          <all>
            <element name="tickerSymbol" type="string"/>
          </all>
        </complexType>
      </element>
      <element name="TradePrice">
        <complexType>
          <all>
            <element name="price" type="float"/>
          </all>
        </complexType>
      </element>
    </schema>
  </types>

  <message name="GetLastTradePriceInput">
    <part name="body" element="xsd1:TradePriceRequest"/>
  </message>

  <message name="GetLastTradePriceOutput">
    <part name="body" element="xsd1:TradePrice"/>
  </message>

  <portType name="StockQuotePortType">
    <operation name="GetLastTradePrice">
      <input message="tns:GetLastTradePriceInput"/>
      <output message="tns:GetLastTradePriceOutput"/>
    </operation>
  </portType>

  <binding name="StockQuoteSoapBinding" type="tns:StockQuotePortType">
    <soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
    <operation name="GetLastTradePrice">
      <soap:operation soapAction="http://example.com/GetLastTradePrice"/>
      <input>
        <soap:body use="literal"/>
      </input>
      <output>
        <soap:body use="literal"/>
      </output>
    </operation>
  </binding>

  <service name="StockQuoteService">
    <documentation>My first service</documentation>
    <port name="StockQuotePort" binding="tns:StockQuoteBinding">
      <soap:address location="http://example.com/stockquote"/>
    </port>
  </service>

</definitions>
```

Todos los detalles de WSDL se pueden consultar en la página web del W3C (<http://www.w3.org/TR/wsdl>).

2.3.4 UDDI (Universal Description Discovery and Integration)

Como se ha comentado anteriormente, antes de que un solicitante de servicio pueda invocar un servicio web aquél debe conocer su interfaz. Para ello, es necesario que proveedor y solicitante del servicio compartan la descripción WSDL del servicio.

Una forma cómoda de llevar a cabo esta compartición de información es mediante un directorio UDDI (Universal Description Discovery and Integration). Un directorio UDDI es un servicio de registro o servicio de directorio donde:

- Los proveedores de servicios publican sus servicios web.
- Y los solicitantes de servicios buscan y encuentran servicios web acordes a sus necesidades.

En definitiva, un directorio UDDI viene a ser como un servicio de páginas amarillas (listado de servicios disponibles) aplicado a los servicios web.

UDDI fue estandarizado inicialmente por un consorcio formado por IBM, Microsoft y Ariba, aunque a este consorcio se han ido uniendo más empresas. El estándar UDDI define básicamente dos aspectos:

- La información a almacenar en un directorio UDDI.
- La forma de acceso a un directorio UDDI. Este acceso es mediante SOAP.

La versión actual del estándar es UDDI 3.0.2, se pueden consultar todos sus detalles en <http://uddi.xml.org/> y <https://www.oasis-open.org/standards#uddiv3.0.2>.

2.4 Sintaxis de XML

Como ya se ha adelantado, XML es el lenguaje de marcado que permite describir información y estructura.

Un documento XML está conformado por:

- Etiquetas y atributos.
- Y datos.

Donde los datos aportan la información y las etiquetas y sus atributos identifican la naturaleza de los datos y aportan su estructura. Por ejemplo, un documento XML sencillo sería como sigue:

```
<?xml version="1.0"?>
<!DOCTYPE libro SYSTEM "libro.dtd">
<libro>
    <titulo>XML Pocket Reference</titulo>
    <editorial>Oreilly</editorial>
    <autores>
        <autor principal="si">Robert Eckstein</autor>
        <autor principal="no">Michel Casablanca</autor>
    </autores>
    <numero_paginas>96</numero_paginas>
</libro>
```

Interesa recalcar aquí que los documentos XML sólo contienen información y estructura (ej. los datos descriptivos de un libro), pero nunca instrucciones de formato (ej. colores, tamaños, tipos de letra, etc.). Esta es una de las diferencias clave entre XML y HTML.

La sintaxis de XML es compleja. No es el propósito del presente tema hacer una revisión exhaustiva de la sintaxis de XML. Para ello, se recomienda al lector revisar las referencias especializadas (W3C; <http://www.w3.org/XML/>).

Si un documento respeta las reglas de sintaxis XML se dice que está bien formado. Si además de estar bien formado el documento es conforme a una DTD o un XML Schema (ver apartado siguiente), se dice que el documento es válido.

2.5 Definición de lenguajes y validación de documentos XML (DTDs y XML Schemas)

XML aporta dos herramientas fundamentales para definir lenguajes basados en XML y, una vez definidos esos lenguajes, validar documentos XML contra su definición. Estas herramientas son:

- DTDs (Document Type Definitions).
- XML Schemas.

Cuando un documento XML es conforme a una DTD o un XML Schema, se dice que el documento es válido. Obsérvese que, para ser válido, el documento debe respetar mucho más que las reglas básicas de sintaxis XML (abrir y cerrar las etiquetas, etc.). Además de respetar todas estas reglas básicas, lo que supone que el documento esté bien formado, el documento debe tener un contenido y una estructura acordes a una DTD o XML Schema.

2.5.1 DTDs (Document Type Definitions)

Las DTDs son el mecanismo original de XML para definir lenguajes XML. Una DTD es un documento de texto que especifica:

- Qué elementos pueden formar parte de un documento XML.
- Qué relación o estructura existe entre esos elementos.
- Qué atributos pueden tener los elementos.
- Qué posibles valores pueden tomar los atributos.
- Etc.

Un ejemplo de DTD, aplicable a documentos XML para el intercambio de información entre editores, se puede observar a continuación (fichero “libro.dtd”):

```
<!ELEMENT libro (titulo, editorial, autores, numero_paginas)>
<!ELEMENT titulo (#PCDATA)>
<!ELEMENT editorial (#PCDATA)>
<!ELEMENT autores (autor+)>
<!ELEMENT numero_paginas (#PCDATA)>
<!ELEMENT autor (#PCDATA)>
```

Según esta DTD, para que un documento XML sea válido debe tener un elemento raíz llamado “libro” que constará de:

- Un elemento "título".
- Un elemento "editorial".
- Un elemento "autores", que a su vez constará de:
- Uno o más elementos "autor".
- Un elemento "numero_paginas".

Dicho de otro modo, una DTD no sólo permite definir lenguajes XML. También vale para comprobar si un documento XML concreto responde o no a una definición, es decir, permite comprobar si un documento es válido.

Para entender mejor el concepto de DTD y documento XML se puede establecer una analogía con otros conceptos informáticos, como son los conceptos de clase y objeto de la Programación Orientada Objetos. Así, una DTD sería equivalente a una clase (porque define un patrón) y un documento XML sería equivalente a un objeto (porque es una instancia del patrón general).

Las DTDs pueden definirse dentro de los propios documentos XML o en ficheros aparte. Por cuestión de claridad (separar el patrón de sus instancias), resulta preferible definir las DTDs aparte.

A su vez, para vincular un documento XML con su DTD existen dos posibilidades. Si la DTD se encuentra disponible en el sistema de ficheros local, entonces la cabecera del documento XML deberá ser algo así:

```
<?xml version="1.0"?>
<!DOCTYPE libro SYSTEM "libro.dtd">
...
```

Por otro lado, si la DTD se encuentra disponible en una URL de Internet, entonces la cabecera del documento XML deberá ser algo así:

```
<?xml version="1.0"?>
<!DOCTYPE libro PUBLIC "http://servidor/libro.dtd">
...
```

Esta segunda opción (DTDs disponibles en Internet) resulta muy interesante, ya que permite compartir entre toda la comunidad Internet definiciones comunes para documentos XML. Dicho de otro modo, facilita la implantación de estándares XML.

Para conocer en detalle la sintaxis de las DTDs se recomienda al lector revisar las referencias especializadas (<http://www.w3.org/TR/2004/REC-xml11-20040204/>; apartado 2.8 Prolog and Document Type Declaration).

2.5.2 XML Schemas

Los XML Schemas son la evolución de las DTDs. Sus objetivos son los mismos que los de las DTDs:

- Definir lenguajes XML.
- Validar documentos XML contra su definición.

La principal diferencia entre XML Schemas y DTDs es que los primeros tienen mayor potencia descriptiva. Por este motivo, los XML Schemas se han impuesto a las DTDs, cayendo estas últimas prácticamente en desuso actualmente. Otras diferencias importantes son:

- Los XML Schemas se describen en XML¹⁰, no necesitando una nueva sintaxis para describirlas. Las DTDs tienen una sintaxis propia, como ya se vio.
- Los XML Schemas permiten definir nuevos tipos de datos. Las DTDs no lo permiten.
- Los XML Schemas soportan herencia entre tipos de datos. Las DTDs no lo soportan.
- Los XML Schemas están basados en namespaces¹¹. Las DTDs no.

Como se puede apreciar, casi todo en los XML Schemas son ventajas. Su principal desventaja es que resultan más complejos que las DTDs. Por este motivo, es previsible que en el futuro se siga haciendo uso de las DTDs, aunque de forma limitada, para aquellas aplicaciones basadas en XML que sean sencillas.

Aunque existen otras especificaciones, aquí se revisará la especificación estándar de XML Schemas, que corresponde al W3C. La versión actual de este estándar es XML Schemas 1.1.

Como ya se ha dicho, un XML Schema es un documento XML. El elemento raíz de este documento XML será `<xs:schema> ... </xs:schema>`¹², ya que lo que hace un XML Schema es definir un esquema o tipo de datos. Esto se puede observar en el siguiente ejemplo de XML Schema (fichero "libro.xsd"), que es equivalente a la DTD del apartado anterior:

¹⁰ Resulta curioso. Los XML Schemas se utilizan para definir el contenido y estructura de documentos XML válidos. Pero, a su vez, los XML Schemas son documentos XML.

¹¹ Los namespaces son una extensión de la sintaxis básica de XML. Los namespaces permiten extender el nombre de las etiquetas para pasar de un nombre sencillo (ej. `<libro>`) a un nombre con prefijo (ej. `<editorial:libro>` o `<biblioteca:libro>`). Estos prefijos sirven para evitar potenciales conflictos de nombres en situaciones en que se combinan documentos XML que provienen de distintas fuentes (ej. documentos XML que provienen de editores de libros y de bibliotecas). Los XML Schemas hacen un uso profuso de namespaces.

¹² Obsérvese como, efectivamente, los XML Schemas utilizan namespaces. El prefijo para XML Schemas es "xs".

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="libro">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="titulo" type="xs:string"/>
        <xs:element name="editorial" type="xs:string"/>
        <xs:element name="autores">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="autor" type="xs:string" minOccurs="1"
maxOccurs="unbounded"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
        <xs:element name="numero_paginas" type="xs:integer"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Según este XML Schema, para que un documento XML sea válido debe tener un elemento raíz llamado "libro" que constará de:

- Un elemento "título" de tipo "string".
- Un elemento "editorial" de tipo "string".
- Un elemento "autores", que a su vez constará de:
 - Uno o más elementos "autor".
- Un elemento "numero_paginas" de tipo "integer".

Finalmente, si se desea que el documento XML original esté referenciado al XML Schema anterior, y no a la DTD, entonces habrá que modificarlo como se indica a continuación:

```
<?xml version="1.0"?>

<libro      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="libro.xsd">

    <titulo>XML Pocket Reference</titulo>

    <editorial>Oreilly</editorial>

    <autores>

        <autor>Robert Eckstein</autor>

        <autor>Michel Casablanca</autor>

    </autores>

    <numero_paginas>96</numero_paginas>

</libro>
```

Si se desea conocer todas las posibilidades de XML Schemas (tipos simples predefinidos, definición de tipos complejos, herencia de tipos, etc.) se deberá consultar las referencias especializadas (<http://www.w3.org/XML/Schema>).

2.6 Procesado de documentos XML (DOM y SAX)

Como ya se ha comentado, uno de los usos principales de XML es el intercambio de datos entre aplicaciones, especialmente cuando éstas son heterogéneas (ej. están programadas en lenguajes de programación diferentes, se ejecutan sobre plataformas diferentes, etc.). Pues bien, este intercambio de datos implica que:

- Una aplicación origen debe generar datos en formato XML
- Y otra aplicación destino debe procesar los datos en formato XML, es decir, debe procesar los documentos XML generados por la aplicación origen¹³.

Ambas aplicaciones, origen y destino, deben compartir una DTD o XML Schema para poder entenderse entre sí. Los documentos XML generados por la aplicación origen deberán ser conformes con esa DTD o XML Schema. Por otro lado, la aplicación destino utilizará esa DTD o XML Schema para validar los documentos XML que tenga que procesar.

Todo esto se puede apreciar en la Figura 9:

¹³ Hay otras situaciones, aparte del intercambio de datos entre dos aplicaciones, en las que puede interesar procesar documentos XML. Por ejemplo, si una aplicación guarda datos o su información de configuración en ficheros XML, entonces tendrá que leer del disco estos ficheros y procesar la información XML.

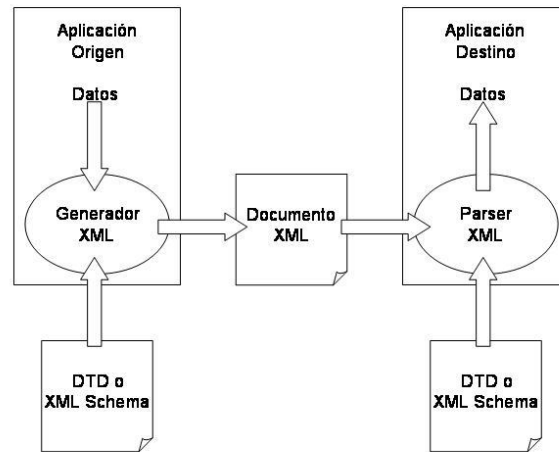


Figura 9. Generación y procesamiento de documento XML

La parte de la aplicación destino que se encarga de procesar los documentos XML se denomina parser XML, y típicamente suele consistir en una librería implementada en un lenguaje de programación (ej. Java). Parsers XML hay muchos, y para muchos lenguajes de programación distintos. En el caso de Java, algunos de los parsers más utilizados son Xerces y Xalan, pero hay muchos otros.

Pues bien, a la hora de procesar documentos XML el parser puede aplicar dos modelos de funcionamiento principales:

- El modelo DOM (Document Object Model)
- El modelo SAX (Simple API for XML).

El modelo DOM (Document Object Model) consiste en que el parser procesa el documento XML completo, elabora una representación en memoria equivalente (un árbol en memoria RAM), y ofrece una serie de funciones para que el programador recorra y modifique el árbol. Esto se puede apreciar en la siguiente Figura 10:

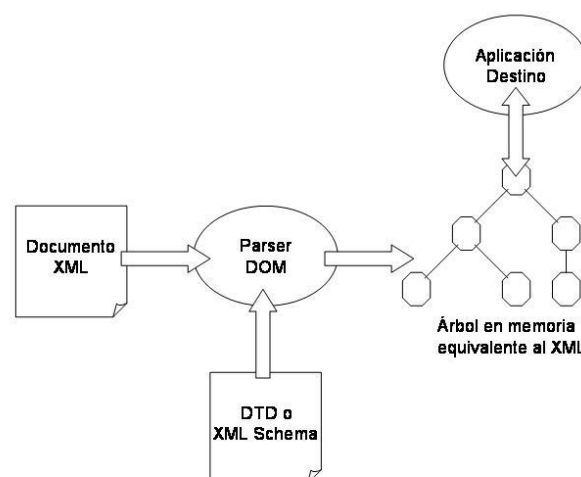


Figura 10. Procesamiento con parser DOM

Por su parte, el modelo SAX (Simple API for XML) consiste en que el parser procesa el documento XML poco a poco (elemento a elemento) y, según lo va procesando, genera eventos (invocaciones de

funciones) que permiten que la aplicación destino recupere la información del documento XML y actúe en consecuencia. Esto se puede apreciar en la siguiente Figura 11:

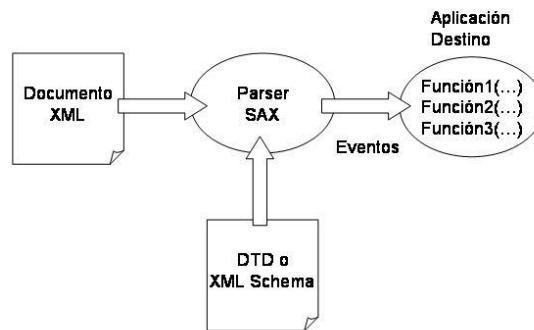


Figura 11. Procesamiento con parser SAX

En principio, no se puede decir que un modelo de procesamiento, DOM o SAX, sea más potente que el otro. Ambos modelos tienen su campo de aplicación y, dependiendo de las circunstancias, se deberá optar por un parser DOM o un parser SAX. Por supuesto, también hay librerías que implementan los dos modelos.

La Tabla 1 se presenta las ventajas y desventajas de cada modelo:

	Ventajas	Desventajas
Modelo DOM	- Facilita el acceso a toda la información del documento XML - Permite la modificación de los datos (en memoria) - Existe una especificación estándar del modelo (interfaz)	- Consume mucha memoria, especialmente para documentos grandes - Existen distintas implementaciones de la especificación DOM
Modelo SAX	- Rápido y ligero - Facilita el acceso a partes concretas del documento XML - El tamaño del documento XML no importa	- No es posible modificar los datos - Resulta difícil implementar búsquedas complejas

Tabla 1 DOM vs. SAX

Posteriormente surgieron otras tecnologías similares como StAX, XOM y JDOM, todas ellas basándose en conceptos de DOM y SAX, siendo muy similares a éstas en varios conceptos.

2.7 Transformación y presentación de documentos XML (XSL y XSLT)

A lo largo del presente tema se ha insistido reiteradamente en que una de las virtudes de XML es que, al contrario que HTML, separa completamente información de presentación. Así, los documentos XML como el ejemplo ya visto:

```
<?xml version="1.0"?>

<libro          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="libro.xsd">

    <titulo>XML Pocket Reference</titulo>

    <editorial>Oreilly</editorial>

    <autores>
        <autor>Robert Eckstein</autor>
        <autor>Michel Casablanca</autor>
    </autores>

    <numero_paginas>96</numero_paginas>

</libro>
```

carecen completamente de instrucciones de formato (ej. colores, tamaños, tipos de letra, etc.).

Pues bien, para dotar a XML de capacidades de formato es para lo que surgió XSL (eXtensible Stylesheet Language). XSL, que es un estándar del W3C, permite complementar la información de un documento XML con una hoja de estilos (instrucciones de formato). Por tanto, XSL es a XML lo que CSS es a HTML.

Así las cosas, las funciones básicas de “información”, “validación” y “presentación” quedan repartidas de la siguiente manera:

- Información: La información o datos se expresa mediante documentos XML.
- Validación: La validación se expresa mediante DTDs o XML Schemas.
- Presentación: La presentación se expresa mediante hojas de estilo XSL.

En realidad, XSL no es un único estándar, sino que es un conjunto de estándares relacionados con la presentación de documentos XML. En este sentido, los estándares de XSL más destacados son:

- XSLT (XSL Transformations): Vale para transformar un documento XML original en otro documento que resulte de la transformación del primero (selección, agrupamiento, reordenación de elementos, etc.). Este documento transformado puede ser XML, HTML, WML, texto plano, etc.
- XPath (XML Path Language): Vale para referenciar partes específicas de un documento XML. Esta referenciación o selección de partes específicas de un documento XML es necesaria tanto para su transformación como para su presentación.
- XSL-FO (XSL Formatting Objects): Vale, propiamente, para la presentación de un documento XML.

A modo de ejemplo, la siguiente hoja de estilo XSLT valdría para transformar el documento XML presentado anteriormente en una página HTML con información equivalente y que, además, incorpora ya instrucciones de formateo:

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:output method="html" version="1.0" encoding="UTF-8" indent="yes"/>

  <xsl:template match="libro">
    <html>
      <head>
        <title>
          <xsl:value-of select="titulo"></xsl:value-of>
        </title>
      </head>
      <body>
        <xsl:apply-templates></xsl:apply-templates>
      </body>
    </html>
  </xsl:template>

  <xsl:template match="titulo">
    <h1>
      <p>
        Titulo: <xsl:apply-templates></xsl:apply-templates>
      </p>
    </h1>
  </xsl:template>

  <xsl:template match="editorial">
    <p>
      Editorial: <xsl:apply-templates></xsl:apply-templates>
    </p>
  </xsl:template>

  <xsl:template match="autores">
    <p>
      Autores: <xsl:apply-templates></xsl:apply-templates>
    </p>
  </xsl:template>

  <xsl:template match="autor">
    <ul>
      <xsl:apply-templates></xsl:apply-templates>
    </ul>
  </xsl:template>

  <xsl:template match="numero_paginas">
    <p>
      Numero Paginas: <xsl:apply-templates></xsl:apply-templates>
    </p>
  </xsl:template>

</xsl:stylesheet>
```

Como se puede observar en el ejemplo anterior, una hoja de estilos XSLT es, en última instancia, un documento XML. Es decir, así como CSS tenía una sintaxis propia, la sintaxis de XSL es XML¹⁴.

El elemento principal de una hoja de estilos XSLT es `<xsl:stylesheet> ... </xsl:stylesheet>`. A su vez, una hoja de estilos está compuesta por una serie de templates o reglas (`<xsl:template> ... </xsl:template>`). Estas reglas se hacen casar contra el documento XML original para generar el documento transformado. Así, en el caso de aplicar esta hoja de estilos al documento XML anterior, resultaría el siguiente documento HTML:

¹⁴ Lo mismo ocurriría con XML Schemas. Un esquema XML es, en última instancia, un documento XML.

```
<html>
  <head>
    <title>XML Pocket Reference</title>
  </head>
  <body>
    <h1>
      <p>
        Titulo: XML Pocket Referente
      </p>
    </h1>
    <p>
      Editorial: Oreilly</p>
    <p>
      Autores:
      <ul>Robert Eckstein</ul>
      <ul>Michel Casablanca</ul>
    </p>
    <p>
      Numero Paginas: 96
    </p>
  </body>
</html>
```

La aplicación de hojas de estilo XSLT a documentos XML se hace mediante parsers XSLT. Un ejemplo de parser XSLT es Xalan de Apache. Por norma general, los parsers XML son también parsers XSLT.

XSLT dispone de templates predefinidos para múltiples funciones: para buscar elementos, para procesamiento condicional, para iteraciones, para ordenaciones, para incluir o importar otras hojas de estilo XSLT, etc. Todos los detalles de XSL y XSLT se pueden consultar en las referencias especializadas (<http://www.w3.org/Style/XSL/> y <http://www.w3.org/TR/xslt>).

2.8 Lenguajes basados en XML

El mundo de XML, como el mundo de la web y HTML, es muy dinámico. Constantemente aparecen en Internet nuevos estándares y tecnologías relacionados con XML.

De hecho, el conjunto de estándares relacionados con XML es ya tan extenso y se reproduce a tanta velocidad que puede llegar a abrumar. Afortunadamente, igual que nacen muchos de estos “estándares”, muchos otros mueren y desaparecen¹⁵ y la inestabilidad es la nota predominante.

Obtener una lista exhaustiva de los estándares XML actualmente en vigor es un trabajo ímprobo y no especialmente útil salvo para propósitos ilustrativos. A lo largo del tema se describen varios de estos lenguajes, y a modo ilustrativo, en los próximos apartados se describirá en más detalle dos campos de especial relevancia donde la adopción de lenguajes basados en XML ha supuesto un mayor despliegue y extensión:

- La sindicación de contenidos/*feeds*, de creciente importancia debido al auge de blogs, foros, medios de comunicación on-line, etc., con RSS y Atom como lenguajes XML que han permitido su extensión.
- La información financiera y tributaria, con un papel destacado de organismos y entidades de la Administración del Estado (Banco de España, CNMV, AEAT, etc.), con XBRL como el

¹⁵ El profesor Tanenbaum decía en tono irónico: “Lo bueno de los estándares es que hay muchos”. Algo así está ocurriendo con XML. Se ha visto tan útil y se ha puesto tan de moda que los “estándares” en torno a XML, normalmente gramáticas para aplicaciones concretas, crecen como setas, dando lugar a una “sopa de letras” imposible de recordar. Algunos de estos estándares llegan a consolidarse, pero muchos otros caen en el olvido en un breve plazo de tiempo.

lenguaje XML que ha ganado la batalla de la estandarización en un sector tan complejo como el financiero.

2.9 Sindicación y XML

El término sindicación es un anglicismo que proviene de "*syndication*", habitual en la terminología anglosajona de los medios de comunicación. Como tal, se desaconseja su uso en español, pero se incorporó rápidamente a la jerga técnica con la popularización de la redifusión de contenidos web.

La "sindicación web", "sindicación de contenidos", o, en términos lingüísticos más recomendables, "redifusión de contenidos", es una forma de distribución de información mediante la cual parte de una página web se pone a disposición para su uso desde otras páginas.

De forma general, los contenidos de las fuentes de sindicación de contenidos suelen codificarse en XML, aunque el formato puede ser cualquiera que pueda transportarse a través de HTTP, como HTML o Javascript. Sin embargo, el binomio sindicación-XML se han convertido en el predominante gracias al ubicuo despliegue de las dos principales familias de formatos de sindicación web: RSS y Atom.

2.9.1 RSS y RDF

RSS es una gramática XML que permite a los sitios web que actualizan su información frecuentemente (ej. periódicos on-line o weblogs) que compartan sus contenidos de forma interoperable con otros sitios o aplicaciones. Esta compartición de contenidos es lo que hemos comentado anteriormente que en terminología web se denomina sindicación.

Pero, ¿de dónde viene la idea de RSS y la sindicación de contenidos?

Si hacemos un poco de historia, cuando el sitio web de Netscape era uno de los sitios más visitados de Internet, surgió la necesidad de hacer más personalizado el portal a sus usuarios, nuevo concepto al que llamaron "My Netscape".

En "My Netscape", los usuarios podían elegir qué contenido poner en su sitio, seleccionando los últimos titulares de noticias, las más recientes actualizaciones de archivos, y así otras diversas opciones. Cada canal, nombre con el que bautizó Netscape a las fuentes de información disponibles, también incluía información mostrando una imagen y una caja de búsqueda para buscar en el sitio del canal. Los servidores de Netscape periódicamente bajaban y actualizaban cada canal desde el sitio que lo proveía, de esta forma la información quedaba automáticamente a disposición del usuario.

En relación al formato que tenían los archivos formateados, Netscape consideró necesario establecer una forma universal para la descripción del contenido, para lo que desarrolló el formato *RDF Site Summary*, que usaba XML y RDF.

RSS 0.9 fue la primera versión liberada por Netscape en marzo de 1999, liberando en julio del mismo año la versión 0.91 incorporando mejoras al formato llamado *<scriptingNews>*, creado por UserLand. Al poco tiempo Netscape discontinuó el desarrollo del formato RSS pero UserLand lo siguió utilizando en My Userland, y RSS se convirtió en *Rich Site Summary* para esta versión.

En agosto del 2000 un grupo llamado RSS-DEV Working Group propuso el RSS 1.0 en una lista de discusión sobre RDF, a lo que UserLand contraatacó en el 2002 y liberó el RSS 2.0 ahora llamado *Really Simple Syndication*.

Por ello, en realidad RSS no es una única gramática XML, sino que en los últimos tiempos ha habido distintas especificaciones, elaboradas por distintas compañías, y todas ellas denominadas "RSS"¹⁶.

Como hemos visto, el acrónimo RSS tiene varios significados distintos, según la especificación de que se trate:

¹⁶ Se observa aquí el ya citado carácter inestable de los (supuestos) "estándares" XML.

- Rich Site Summary (RSS 0.91).
- RDF Site Summary (RSS 0.9 y RSS 1.0).
- Really Simple Syndication (RSS 2.0).

En cualquier caso, la idea subyacente a RSS es siempre la misma: syndicar contenidos de forma interoperable.

Los archivos RSS comúnmente se llaman *feeds* RSS o canales RSS y contienen un resumen de lo publicado en el sitio web de origen. Se estructuran en uno o más ítems, donde cada ítem consta de un título, un resumen de texto y un enlace a la fuente original en la web donde se encuentra el texto completo. Además puede incluir información adicional como el nombre del autor o la fecha y la hora de publicación del contenido.

Por tanto, cualquier fuente de información susceptible de poder ser parcelada en ítems (los mensajes de un foro, por ejemplo) pueden distribuirse utilizando RSS.

Las aplicaciones que agregan contenidos provenientes de distintas fuentes se denominan agregadores RSS.

Finalmente, en relación a RDF (*Resource Description Framework*) es importante considerar que su aplicación va más allá de la sindicación de contenidos y que su relación con la misma viene dada por el uso que hicieron desde Netscape del mismo para las primeras versiones de RSS.

RDF es una de las tecnologías esenciales de la Web Semántica, que persigue la idea de añadir metadatos semánticos a la Web, informaciones adicionales —describiendo el contenido, el significado y la relación de los datos— dadas de manera formal, de forma que sea posible evaluarlas de forma automática sin intervención humana.

El objetivo es mejorar la Web ampliando la interoperabilidad entre los sistemas informáticos reduciendo la necesaria mediación de operadores humanos.

2.9.2 Atom

A raíz de la confusión de versiones de RSS netamente incompatibles unas con otras, en 2003, IBM, Google y otras empresas de *hosting* crearon una nueva tecnología para syndicar contenido, que tras varios cambios de nombre finalmente ha sido bautizado como *Atom*.

Para evitar los problemas de su predecesor, el formato de sindicación fue publicado en 2005 como un “estándar propuesto” (*proposed standard*) del IETF en la RFC 4287, y en la actualidad ha surgido tras él el protocolo *Atom Publishing Protocol* (APP) un protocolo bastante simple basado en HTTP pensado para crear y actualizar recursos web, publicado en octubre de 2007 como “estándar propuesto” (*proposed standard*) del IETF en la RFC 5023.

El éxito de Atom vino de la mano de la adopción de mismo por parte de Google y sus servicios, como Blogger y Google News.

Un ejemplo del aspecto que tiene una fuente Atom se muestra a continuación.

```
<?xml version="1.0" encoding="utf-8"?>
<feed xmlns="http://www.w3.org/2005/Atom">
  <title>Example Feed</title>
  <subtitle>A subtitle.</subtitle>
  <link href="http://example.org/" />
  <updated>2015-05-10T13:13:23Z</updated>
  <author>
    <name>Juan Palomo</name>
    <email>juanpalomo@example.com</email>
  </author>
  <id>urn:uuid:60a76c80-d399-11d9-b91C-0003939e0af6</id>
  <entry>
    <title>Atom-Powered Robots Run Amok</title>
    <link href="http://example.org/2003/12/13/atom03" />
    <id>urn:uuid:1225c695-cfb8-4ebb-aaaa-80da344efa6a</id>
    <updated>2015-05-09T13:14:43Z</updated>
    <summary>Texto variado.</summary>
  </entry>
</feed>
```

2.9.3 Estado actual

En los últimos años el uso de la sindicación no ha continuado creciendo como se podía esperar. El consumo de contenido digital se ha trasladado a otras herramientas tales como redes sociales o herramientas como FlipBoard, todas ellas herramientas no basadas en la sindicación como se ha descrito en el presente tema.

Sin embargo, eso no implica que sea una tecnología que no se utilice hoy en día, utilizándose en diversos ámbitos como el denominado *Broadcatching*, que consiste en la combinación de un sistema de sindicación y otro de descarga convencional o p2p para descargar material digital, tal y como se realiza con los PodCasts, extendiéndose a otros ámbitos como el BitTorrent¹⁷.

2.10 XML y la información financiera: XBRL

XML se ha convertido en la lengua franca de Internet, y como hemos visto, su extensibilidad para crear nuevos lenguajes permite crear sintaxis para diferentes propósitos.

Una de estas nuevas sintaxis, XBRL, es un paso adelante en la estandarización y automatización de los procesos de transmisión y recepción de la información financiera y de negocio, un nuevo lenguaje basado en XML que permite que el intercambio de información contable, financiera y tributaria entre empresas, organismos y la propia Administración Pública.

Si la asimilación de un estándar por parte de la industria es siempre un proceso largo y tortuoso, la adopción de un estándar abierto es una buena noticia para todos, y más aún si cabe si además el consenso se produce en un sector tan complejo como el de las finanzas y a escala mundial.

Hay que tener en cuenta que la información financiera, pese a lo que a priori puede pensarse, es distinta en los diferentes países debido a la diversidad de prácticas contables que se utilizan en todo el mundo, los diferentes mercados de valores, etc. Sin embargo, la necesidad de transmitir cierta información homogénea es una necesidad en todos ellos.

El lenguaje *eXtensible Business Reporting Language* o XBRL es un formato estándar basado en XML que permite el intercambio de información financiera de las empresas (balance, cuenta de pérdidas y ganancias, *cash flow*, etc.). Es de carácter abierto no propietario, y por tanto, libre de licencias por uso.

¹⁷ <http://www.eweek.com/c/a/Messaging-and-Collaboration/BitTorrent-and-RSS-Create-Disruptive-Revolution>

Las principales ventajas de XBRL son:

- Inmediatez en la transmisión de la información a los accionistas, los analistas y al mercado en general, lo que redundará en una mayor transparencia
- Reducción de errores, al no tenerse que reintroducir la información
- Reducción de costes, derivada de la automatización de procesos
- Flexibilidad, al poderse exportar la información en cualquier formato existente
- Interoperabilidad entre los distintos sistemas informáticos.

2.10.1 XBRL en España

En España, el XBRL es introducido en 2001. Asumido el papel regulador por parte del Banco de España, es éste quien lidera el impulso de la utilización de XBRL en España a través del a Jurisdicción Española XBRL, en la que participan también organizaciones como la CNMV (uno de los socios más destacados por parte de nuestro país), Registradores de España y AECA.

A nivel del sector público la Agencia Tributaria ha mostrado su interés en el uso de este estándar para la tramitación telemática de impuestos como el Impuesto de Sociedades, la obtención de informes financieros para estudio de inspecciones tributarias, etc.

No hay que olvidar que aunque originalmente estaba diseñado para *reporting* financiero, XBRL permite integrar información fiscal y potencia la evolución natural de la representación XML aplicada al entorno de los tributos, mejorando la integración de datos a nivel internacional, clarificando la comunicación y reduciendo costes.

Los retos para la Agencia Tributaria se encuentran en la definición de taxonomía de impuestos como el impuesto de sociedades, la integración de salidas XBRL de los sistemas de información de las empresas hacia la AEAT así como la integración de la información pública de la CNMV y del Banco de España, todo ello para disminuir la carga fiscal de los contribuyentes para el cumplimiento voluntario de sus obligaciones.

El impulso definitivo a XBRL, no obstante, vino de la mano de la moción por la que el Senado “insta al Gobierno a impulsar el estándar abierto XBRL”, moción aprobada el 20 de junio de 2006 y que se puede consultar en la siguiente dirección. Esta moción solicitaba al Gobierno su implicación en el liderazgo, estandarización y homogeneización de la implantación de XBRL. El Senado consideró especialmente importante la adopción de un estándar abierto que pudiera ser utilizado por más de 8.000 ayuntamientos, así como 17 comunidades autónomas, todas las provincias, consejos y cabildos insulares, varios ministerios y otros organismos con competencias en el sistema financiero español.

La implantación de la taxonomía LENLOC, basada en el estándar XBRL, responde al concepto de cooperación en materia de interoperabilidad de sistemas y aplicaciones, que propugnan la ley 39/2015 y la ley 40/2015, por cuanto garantiza su inmediata adaptabilidad a diferentes sistemas de información. El archivo o instancia XBRL-LENLOC es generado automáticamente a partir de las distintas bases de datos contables de las corporaciones locales, y permite una explotación automatizada de su contenido desde cualquier sistema¹⁸.

El estándar XBRL permite a la Secretaría General de Coordinación Autonómica y Local (SGCAL) mejorar la transparencia en la rendición de los datos presupuestarios del Sector Público Local, ofreciendo a todas las entidades locales la posibilidad de rendir sus datos presupuestarios en formato XBRL de acuerdo con las taxonomías LENLOC, PENLOC y TRIMLOC definidas por la propia SGCAL. Se puede encontrar más información en el siguiente enlace: <http://www.minhap.gob.es/es-ES/Areas%20Tematicas/Administracion%20Electronica/OVEELL/Paginas/LENLOC.aspx>

¹⁸

http://administracionelectronica.gob.es/pae_Home/dms/pae_Home/documentos/Estrategias/pae_Tecnimap/pae_TECNIMAP-2010/pae_COM_2010-Eficiencia_y_sostenibilidad/10eficiencia.pdf

No en vano, más del 80% de las empresas españolas realizan su depósito anual de cuentas usando este estándar y las empresas reguladas por la CNMV o por el Banco de España realizan sus declaraciones anuales usando la especificación, por poner ejemplos significativos entre otros en uso actualmente¹⁹.

Podemos ver algunos casos de éxito nacionales de esta tecnología en la sede española de los promotores del estándar²⁰.

2.11 Desventajas de XML y alternativas

Como se ha comentado a lo largo del tema XML ofrece muchas ventajas. Sin embargo, XML adolece de ciertas deficiencias que han propiciado el uso de lenguajes alternativos en los últimos años.

Las principales desventajas vienen motivadas por el uso de XML en el mundo web, y se pueden resumir en tres:

- XML introduce mucha sobrecarga. Es un lenguaje que no está pensado para transmitir grandes cantidades de datos. La gran sobrecarga que añade y el no ser un lenguaje binario requieren un mayor tamaño para transmitir la misma información que con otros lenguajes más livianos.
- La computación de XML no es lo suficientemente ligera. Para realizar uso de todo su potencial se requiere un cálculo que no siempre es asumible. Un servidor que recibe millones de peticiones diarias probablemente no utilizará XML como lenguaje de intercambio de información debido a la sobrecarga que introduce.
- Es más complejo para tareas pequeñas. Comenzar a utilizar XML para una pequeña aplicación web supone una sobrecarga que en ocasiones no ofrece ningún beneficio.

Por estas razones y otras razones en el mundo web se ha comenzado a realizar uso de JSON (acrónimo de JavaScript Object Notation). JSON es un subconjunto de la notación literal de objetos de JavaScript que no requiere el uso de XML, y que se utiliza frecuentemente como alternativa a éste. Su eficiencia y facilidad de uso en JavaScript han sido claves para conseguir generalizar su uso como alternativa a XML. Sin embargo, esto no implica que no sean compatibles, siendo frecuente encontrar XML y JSON en una misma aplicación. En el siguiente enlace (<http://www.json.org/xml.html>) podemos ver una comparativa más extensa entre ambos lenguajes.

2.12 Normativa aplicable

XML es una tecnología ampliamente utilizada en la administración. Es importante resaltar el hecho de que la Administración fomenta la representación semántica de la información y facilita la reutilización de vocabularios mediante el Centro de Interoperabilidad Semántica (CISE) de la Administración previsto en el artículo 10, apartado 3 del Real Decreto 4/2010. Los modelos de datos a publicar en el Centro de Interoperabilidad Semántica (CISE) se ajustan a la estructura de intercambio definida en el anexo de la resolución, especificando para los activos semánticos el uso del formato XSD (XML Schema Definition).

En esta línea el Esquema Nacional de Interoperabilidad (ENI)²¹, regulado por el Real Decreto 4/2010, de 8 de enero, establece el conjunto de criterios y recomendaciones que deberán ser tenidos en cuenta por las AA.PP. para la toma de decisiones tecnológicas que garanticen la interoperabilidad, creando así las condiciones necesarias para garantizar el adecuado nivel de interoperabilidad técnica, semántica y organizativa de los sistemas y aplicaciones empleados por las Administraciones públicas.

¹⁹ <http://www.xbrl.es/es/xbrl-una-perspectiva-de-10-anos/>

²⁰ <http://www.xbrl.es/es/socios/casos-de-exito/>

²¹ Se puede encontrar más información en el Centro de Transferencia Tecnológica (CTT) <http://administracionelectronica.gob.es/es/ctt/eni>

En él se establece el desarrollo de la serie de Normas Técnicas de Interoperabilidad (NTIs)²² que son de obligado cumplimiento por parte de las Administraciones públicas.

En la resolución de 19 de febrero de 2013, de la Secretaría de Estado de Administraciones Públicas, por la que se aprueba la Norma Técnica de Interoperabilidad de Reutilización de recursos de la información hace referencia al formato de los documentos y recursos de información reutilizables, indicando que los documentos y recursos de información reutilizables puestos a disposición pública, los metadatos y los servicios asociados a los mismos utilizarán estándares abiertos y se ceñirán a lo establecido en la Norma técnica de interoperabilidad de catálogo de estándares. Se seleccionarán preferentemente formatos que ofrezcan representación semántica de la información, con el fin de facilitar una mejor comprensión de la información representada y su tratamiento automatizado. Si los formatos elegidos lo permiten, se priorizará el uso de esquemas o vocabularios internacionalmente reconocidos para representar la información. Todas estas referencias son satisfechas por XML y lenguajes basados en éste, algo que el Catálogo de estándares pone de manifiesto como se verá a continuación. Además, esta resolución define en su anexo VI el Modelo de plantilla RDF/XML de definición de catálogos y registros.

La Norma Técnica de Interoperabilidad de Catálogo de estándares establece un catálogo formado por un conjunto mínimo de estándares que satisfacen lo previsto en el artículo 11 del Real Decreto 4/2010, de 8 de enero, y que dan soporte al resto de Normas Técnicas de Interoperabilidad. En este catálogo podemos encontrar una gran cantidad de lenguajes basados en XML, que utilizan éste como soporte, así como el propio XML. Varios de ellos se describen en más detalle en otros temas, por lo que sólo se enumerarán a modo de ejemplo de la importancia de XML en la administración.

XAdES, XML-DSig, ETSI TR 102 038, GML, WFS, WMS, Strict Open XML (.docx, .xlsx, .pptx), XHTML, SOAP, UDDI, WSDL, WS-Security, ASN.1, CODICE, Facturae, SCSP, Sistemas de Información Común de Registros de Entrada y Salida (SICRES), Data Catalog Vocabulary (DCAT), OWL, RDF, RDF-A, SKOS (Simple Knowledge Organization System), PREMIS, MoReq, Turtle, XML, XSD.

No se encuentran pues, recogidos los DTD ni JSON. Como se puede observar son muchas las tecnologías que utilizan o están basadas en XML en la administración en comparación con JSON, si bien esto no quiere decir que no se utilice JSON en aplicaciones web donde la comunicación entre cliente y servidor, siendo esto algo transparente al usuario.

Un ejemplo de la utilización de XML y XSD es el del Expediente Electrónico, Norma Técnica de Interoperabilidad aprobada por la Resolución de 19 de julio de 2011 (BOE de 30 de julio), de la Secretaría de Estado para la Función Pública. En tal resolución²³ se pueden encontrar los esquemas XML para el intercambio de expedientes electrónicos (XSDs), también disponibles en el Centro de Transferencia Tecnológica (CTT)²⁴, junto con una guía de aplicación.

²² Se pueden descargar y encontrar más información en el Centro de Transferencia Tecnológica (CTT)

http://administracionelectronica.gob.es/pae/Home/pae_Estrategias/pae_Interoperabilidad_Inicio/pae_Normas_tecnicas_de_interoperabilidad.html

²³ Disponible por ejemplo aquí: http://www.boe.es/diario_boe/txt.php?id=BOE-A-2011-13170

²⁴

http://administracionelectronica.gob.es/pae/Home/pae_Estrategias/pae_Interoperabilidad_Inicio/pae_Normas_tecnicas_de_interoperabilidad.html#EXPEDIENTEELECTRONICO

3 Navegadores web y compatibilidad con estándares.

3.1 Navegadores web

Los navegadores web han ido ganando importancia hasta convertirse en las aplicaciones que más usamos cotidianamente y en las que más tiempo invertimos, ya que se han convertido en la puerta de acceso a toda una serie de servicios que se han vuelto imprescindibles: correo electrónico, agenda, redes sociales, prensa digital, vídeos on-line, mapas, compra electrónica, viajes, consultas en foros o blogs, banca electrónica, entradas para eventos, trámites con la administración, consulta y entrega del IRPF, almacenamiento en la nube, descargas, trabajo colaborativo, trámites o compra.

3.1.1 Historia

El primer navegador web fue desarrollado por Tim Berners-Lee, en la CERN, a finales de 1990 y principios de 1991; el navegador web llamado WorldWideWeb era bastante sofisticado y gráfico, pero solo funcionaba en estaciones NeXT.

Posteriormente surgió el navegador Mosaic, que funcionaba inicialmente en entornos Unix sobre XFree86 (X11), fue el primero que se extendió debido a que pronto el National Center for Supercomputing Applications (NCSA) publicó versiones para Windows y Macintosh.

Sin embargo, Netscape Navigator al poco tiempo entró en el mercado y rápidamente superó en capacidades y velocidad al Mosaic. Este navegador tuvo la ventaja de funcionar en casi todos los sistemas Unix, y también en entornos Windows.

Internet Explorer (anteriormente Spyglass Mosaic) fue la apuesta tardía de Microsoft para entrar en el mercado, pero consiguió desbancar a Netscape Navigator entre los usuarios de Windows, debido a la integración del navegador con el sistema operativo y al hecho de que era gratuito, mientras que Netscape tenía costo, llegando a poseer cerca del 95% de la cuota de mercado. Netscape Communications Corporation liberó el código fuente de su navegador, naciendo así el proyecto Mozilla.

Finalmente, Mozilla (Mozilla Application Suite) fue reescrito desde cero tras decidirse a desarrollar y usar como base un nuevo conjunto de widgets multiplataforma basado en Extensible Markup Language (XML) llamado XUL y esto hizo que tardara bastante más en aparecer de lo previsto inicialmente, apareciendo una versión 1.0 de gran calidad y para muchísimas plataformas a la vez el 5 de junio de 2002.

El 7 de enero de 2003, Apple lanzó al mercado el navegador web Safari. Este navegador se hace con casi la totalidad del mercado de los equipos Mac, debido a su velocidad y gran cantidad de actualizaciones. Asimismo, Safari también entra al mercado del sistema operativo Windows.

A finales de 2004 aparece en el mercado Mozilla Firefox, una rama de desarrollo de Mozilla que pretende hacerse con parte del mercado de Internet Explorer. Se trata de un navegador más ligero que su hermano mayor.

El 2 de septiembre de 2008, Google Chrome aparece en el mercado. Es el navegador web desarrollado por Google y compilado con base en componentes de código abierto como el motor de renderizado de WebKit y su estructura de desarrollo de aplicaciones (framework). Está disponible gratuitamente bajo condiciones de servicio específicas. El nombre del navegador deriva del término usado para el marco de la interfaz gráfica de usuario ("chrome"). En diciembre de 2011, Chrome superó a Internet Explorer 8.0 como el navegador más utilizado a nivel mundial.

3.1.2 Conceptos

Aunque se han introducido ya al comienzo del tema o pueden resultar conocidos, vamos a recordar en este apartado alguno de los conceptos esenciales que es necesario tener claros para continuar.

Un **navegador web** es una aplicación que permite el acceso a internet, interpreta la información de archivos etiquetados en HTML y los presenta en pantalla según las directrices de presentación codificadas en una hoja de estilos CSS (del inglés Cascading Style Sheet, u hoja de estilo en cascada), permitiéndonos interactuar con su contenido.

Los documentos presentados en el navegador denominados páginas web, poseen **hipervínculos** que permiten enlazar de un documento a otro. Se denomina **navegar** al acto de seguir los enlaces de una página a otra.

Una **URL** es la cadena de caracteres con la que se asigna una dirección única a todos y cada uno de los recursos de información disponibles en internet. Para entendernos, el URL es la dirección de internet, que permite al navegador encontrar ese recurso en un servidor determinado y mostrarlo de forma adecuada.

El **protocolo DNS** asigna nombres a direcciones IP (es lo que permite encontrar un servidor tecleando un nombre).

Un **puerto** es una interfaz a través de la cual pueden enviarse y recibirse datos para comunicarse con otro programa o aplicación a través de una red telemática. Podemos comprenderlo más gráficamente si pensamos en un puerto como en una pasarela, que puede estar abierta o cerrada, por la que circulan los datos.

3.1.3 Funcionamiento de un navegador

La función básica de un navegador es la de visualizar unos documentos denominados páginas web que pueden estar compuestas tanto de texto como de todo tipo de elementos multimedia, y que usualmente están almacenados en unos ordenadores remotos conectados a internet con un software especial para permitir el acceso, (denominados servidores web), y a los que se accede mediante un protocolo (conjunto de reglas y normas que permiten la intercomunicación de dispositivos) conocido como HTTP (Hypertext Transfer Protocol, o Protocolo de transferencia de hipertexto).

En la Figura 12 se muestra el flujo de intercambio de mensajes que se produce durante la navegación entre los principales elementos: el navegador, el servidor DNS y el servidor web.

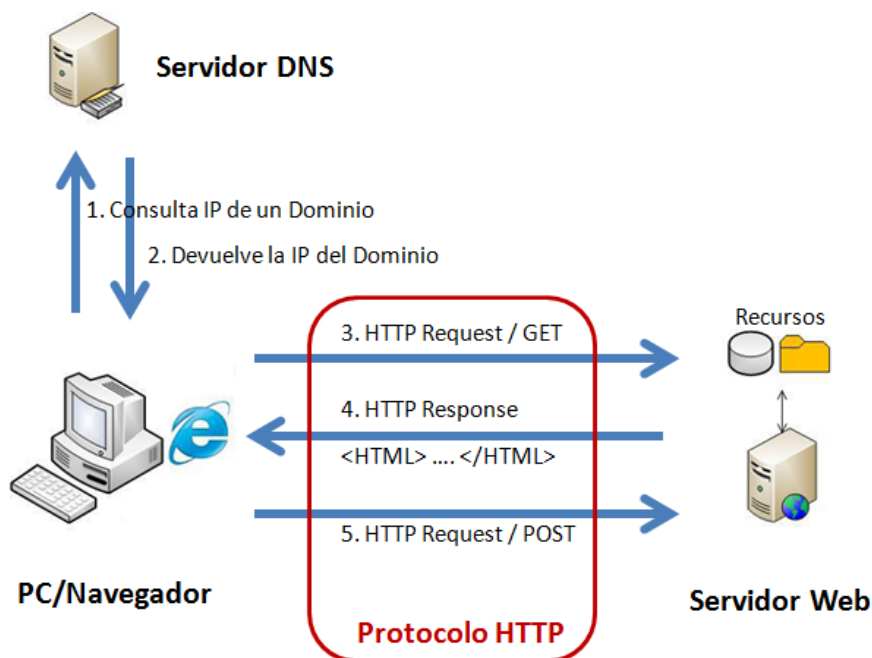


Figura 12. Intercambio de mensajes en la navegación

Además del protocolo HTTP, la mayoría de navegadores soporta otros protocolos adicionales, como FTP (File Transfer Protocol, o Protocolo de transferencia de ficheros) o HTTPS (Hypertext Transfer Protocol Secure, o Protocolo seguro de Transferencia de hipertexto).

Todo este conjunto de protocolos de aplicación se apoyan en el TCP (Transmission Control Protocol, o Protocolo de Control de Transmisión), que es el responsable de crear la conexión entre el ordenador del usuario y el servidor y de garantizar que los datos serán entregados en destino sin errores y en un orden idéntico al que fueron transmitidos. También gestiona los distintos puertos de comunicación de cada máquina.

Cada máquina conectada a internet tiene una dirección IP (Internet Protocol, o Protocolo de Internet) única que la identifica. Cuando escribimos una dirección web o URL (Uniform Resource Locator, o Localizador uniforme de recursos), otras máquinas denominadas servidores DNS (Domain Name System, o Sistema de Nombres de Dominio) traducen o asignan el nombre de dominio que hemos escrito (por ejemplo, www.inap.es) a su dirección IP (que en el caso del INAP es 185.73.174.129), lo que permite al navegador encontrar el servidor de destino y pedirle el recurso solicitado.

El servidor proporcionará la información al navegador, que la descargará y renderizará de acuerdo con las directrices de aspecto y formato contenidas en una hoja de estilo CSS (Cascade Style Sheet, u hoja de estilos en cascada).

El ciclo de solicitud-respuesta HTTP que permite a la máquina del usuario visualizar el contenido almacenado en un servidor remoto se puede resumir en los siguientes pasos:

1. Escribimos la URL en la barra de direcciones del navegador.
2. Un servidor de nombres DNS busca la página que hemos escrito, averigua su dirección IP y la devuelve al navegador.
3. El ordenador o dispositivo del usuario envía una solicitud al servidor identificado por la dirección IP y espera respuesta.
 - a. Si todo va bien, el servidor envía un breve mensaje de retorno al navegador indicando que todo está correcto (transacciones HTTP), seguido del contenido de la propia página web.
 - b. Si hay problemas, el servidor envía al navegador un código de error HTTP que identifica la naturaleza del problema (el más conocido es el error 404: "no se encuentra la página")

3.1.4 Componentes de un navegador

Podemos decir que las principales partes de un navegador web son:

1. La Interfaz de usuario
2. Un layout engine o motor de renderizado (el verdadero núcleo del navegador)
3. Un JavaScript engine o intérprete JavaScript

En la Figura 13 se muestra los principales módulos que comúnmente tienen un navegador actual y cuál es el intercambio de mensajes habitual entre ellos.

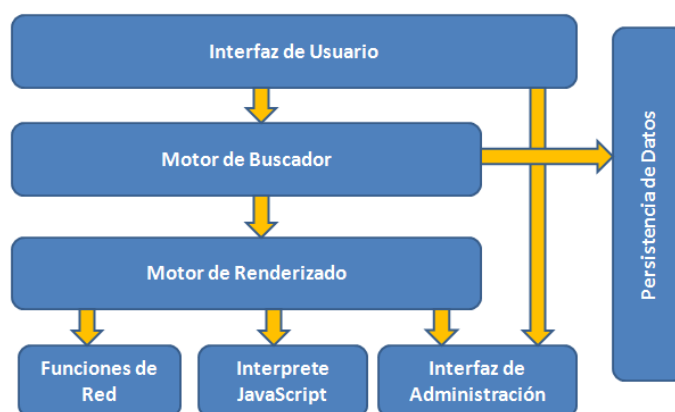


Figura 13. Componentes de un Navegador

A continuación se analizarán cada uno de los elementos más importantes de un navegador entrando en algunos detalles relevantes de los mismos.

3.1.4.1 Interfaz Gráfica de Usuario

En los navegadores, como en cualquier aplicación de software, la interfaz gráfica de usuario o GUI (del inglés Graphic User Interface) es el medio por el cual interactuamos con la aplicación.

Su función es que podamos desarrollar la comunicación con la aplicación de la forma más fácil y cómoda, y que las funciones más usadas tengan acceso preferente.

Cada navegador tiene su propia interfaz de usuario, con distinta organización de menús y submenús, distinta forma de representar las pantallas, distinta organización de las funcionalidades.

3.1.4.2 Motor de Renderizado

Es el componente más importante de cualquier navegador, ya que el motor de renderizado toma el contenido etiquetado con HTML y la información de estilos (maquetación) contenida en las hojas de estilo en cascada o CSS (del inglés Cascade Style Sheet) y las combina para mostrar el contenido ya formateado.

Para entendernos, el motor de renderizado pinta el contenido en una ventana, que es mostrada al usuario o, en su caso, impresa.

A continuación, vamos a hacer un repaso a los principales motores de renderizado:

1. Webkit

WebKit, es un motor de renderizado de código abierto para navegadores web, desarrollado por Apple sobre la base del motor de renderizado KHTML del navegador Konqueror (perteneciente al proyecto KDE: una comunidad que desarrolla software libre).

Webkit incluye dos frameworks o procesadores de más bajo nivel:

- **WebCore**, que es el analizador sintáctico y motor de renderizado HTML
- **JavaScriptCore**: intérprete de JavaScript basado en KJS (KDE's JavaScript, o intérprete de JavaScript del proyecto KDE)

Entre sus funciones está la de gestionar con el servidor web la obtención y renderización de las páginas web, descargar archivos y administrar plugins.

WebKit es el layout engine de Safari. Lo ha sido también de Chrome, pero en abril de 2013 Google anunció que dejaba de colaborar en el proyecto WebKit para desarrollar su propio motor. Webkit está presente también en las versiones móviles de Safari en iOS (iPad, iPhone), y en el navegador nativo de Android, por lo que WebKit es el motor de renderizado líder absoluto entre los navegadores modernos, tanto de sobremesa como móviles.

Por su parte, Opera abandonó en febrero de 2013 el desarrollo de su motor Presto y anunció que pasaría a usar WebKit.

Actualmente, Webkit es el motor de renderizado que soporta mayor número de las características de HTML5 y CSS3.

2. Gecko

Es un motor de renderizado de código abierto, diseñado para soportar los estándares web y originalmente desarrollado por Netscape, que posteriormente lo cedió a la comunidad, y actualmente lo gestiona la Fundación Mozilla. Escrito en el lenguaje de programación C++, es el motor que utiliza, entre otros, del navegador Firefox.

Gecko se implementa como una plataforma para aplicaciones multiplataforma (que pueden correr en cualquier sistema operativo), es decir, que permiten la ejecución de aplicaciones sobre su engine (motor), cuya interfaz de usuario viene definida por el lenguaje XUL (XML-based User Interface)

Language, o lenguaje basado en XML para interfaz de usuario), mientras que el comportamiento y la apariencia de las aplicaciones XUL viene definido por el lenguaje XBL (eXtensible Bindings Lenguaje, o Lenguaje extensible de asociaciones). Las extensiones de Firefox, por ejemplo, tienen una interfaz definida por XUL mientras que su comportamiento es manejado por XBL.

3. Trident y Tasman

Trident es el motor de renderizado usado por Microsoft Internet Explorer para windows a partir de su version 4 (en octubre de 1997). Implementa una interfaz COM (Component Object Model, o Modelo de Objeto de Componente) que soporta toda una serie de tecnologías como OLE, ActiveX, COM+ y DCOM.

Tasman es el nombre del motor de renderizado usado por Internet Explorer para Mac OS X, aunque el desarrollo de IE para Mac fue cancelado en 2001.

4. Blink

Comenzó a desarrollarse como una bifurcación (o fork) del componente WebCore del motor WebKit en abril de 2013, como parte del proyecto Chromium, de la mano de Google y Opera.

Google anunció que cesaba en su apoyo al proyecto WebKit original para centrarse en Blink, que en su actual estado de desarrollo (verano de 2013) permanece relativamente similar a WebCore. Google justifico el cambio por su intención de optimizar el código base de Chrome/Chromium y orientar su desarrollo exclusivamente a la web.

5. Presto

Ha sido el motor de renderizado del navegador Opera, lanzado en febrero de 2003 con la versión 7.0 del browser. Presto destaca por su soporte de DOM (Documents Object Model, o Modelo de Objetos de Documento), DHTML (HTML Dinámico) y CSS2 (Hojas de estilo en Cascada).

Opera ha destacado desde sus inicios por su amplio soporte multiplataforma: windows, Mac OS X, Linux, Symbian (el sistema operativo de los primeros smartphones de Nokia) o J2ME/JME (Java Micro Edition, un subconjunto de la plataforma Java para dispositivos con recursos restringidos, y muy empleado en los teléfonos móviles hace unos años).

Opera anunció en 2013 que dejaba Presto por WebKit, tanto en las versiones móviles como en las de escritorio, y que en futuras versiones estudiará el uso de Blink.

3.1.4.3 Motores JavaScript

Un motor JavaScript (JavaScript Engine) es la parte del navegador que interpreta y ejecuta el código escrito en el lenguaje de programación JavaScript.

Los motores JavaScript son exclusivos de cada navegador, y constituyen otro elemento fundamental para la velocidad a la que cada navegador es capaz de interpretar las instrucciones y realizar el renderizado de la página a cargar. En resumen, la combinación motor de renderizado + motor de JavaScript es la que determina la velocidad a la que cada navegador carga las páginas web.

Los motores de JavaScript adquirieron importancia a raíz de la aparición de Google Chrome en 2008, que literalmente barrió a la competencia en velocidad de ejecución. Pronto Mozilla y WebKit reaccionaron, entablándose una auténtica carrera.

Los motores JavaScript evolucionan tan rápido que es difícil seguir su actual estado de desarrollo. El motor usado por Google es el motor de código abierto V8, Mozilla sigue evolucionando su Monkey, Microsoft potencia Chakra para su Internet Explorer, mientras que en Safari/Apple evolucionan su misterioso motor Nitro.

3.1.4.4 Complementos

Un complemento es una aplicación secundaria o subordinada a otra aplicación principal, a la que aporta funcionalidades nuevas y específicas, y con la que se comunica a través de la API (Application Programming Interface o Interfaz de programación de aplicaciones). Se considera que la API es una interfaz estándar que permite a terceros desarrollar todo tipo de aplicaciones que interactúan de forma estable con la aplicación principal.

A los complementos también se los conoce como plug-ins, add-ons, conectores o extensiones.

En los navegadores web, los complementos suelen extender o ampliar las funcionalidades del navegador para visualizar distintos contenidos multimedia o interactivos, como PDFs, audio o vídeo, o añadir funciones más complejas, como Java.

Un ejemplo que fue muy popular de plugin es Adobe Flash Player, que permite visualizar aplicaciones interactivas y también visualizar vídeos en el conocido formato FLV (Flash Video), como los de YouTube. Ahora este plugin ha perdido mucha popularidad a favor de soluciones compatibles con HTML5.

Por otra parte, los complementos pueden presentar algunos inconvenientes:

- Pueden ralentizar el funcionamiento del navegador, causando incluso su cuelgue total.
- Pueden presentar problemas de seguridad o vulnerabilidades.
- La experiencia de usuario puede diferir entre distintas plataformas.
- El rendimiento puede depender de la potencia de la máquina.
- Pueden causar incompatibilidades con otros complementos instalados.

3.1.5 Navegación segura, privada y anónima

3.1.5.1 Navegación segura

El protocolo HTTPS (Hypertext Transfer Protocol Secure, o protocolo HTTP seguro) garantiza que las sesiones de navegación están cifradas, por lo que la transferencia de datos es segura.

Es posible saber que te encuentras en una sesión de navegación segura cuando, en la barra de navegación, aparecen las siglas https.

Resulta fundamental que te encuentres dentro de una sesión segura cuando introduzcas o manejes datos sensibles, como datos bancarios, fiscales, administrativos o de compras.

Este tema se analiza más profundamente en otro tema, pero no está de más mencionarlo aquí justo antes de hablar de la navegación privada.

3.1.5.2 Navegación privada

Con la navegación privada, el navegador no deja en el ordenador ningún rastro de las páginas que visita (cookies, caché e historial). Sin embargo, hay que tener en cuenta que este tipo de navegación no oculta la IP ni proporciona navegación anónima real.

La navegación privada resulta útil en algunos de estos casos:

- Para abrir sesiones paralelas de una misma aplicación desde un mismo ordenador: por ejemplo, podemos tener varias cuentas de GMail abiertas, en lugar de tener que cerrar una sesión y abrir otra, o abrir sesión en otro navegador.
- Para mantener la privacidad de cada usuario en ordenadores compartidos y, evitar, por ejemplo, que datos personales o privados queden expuestos inadvertidamente (formularios, claves, etc).
- Para visitar páginas sospechosas o que generen poca confianza: así se evita que se pueda instalar 'malware' (aplicaciones dañinas) por medio de cookies.

Si bien cada navegador realiza esta función a su manera, en términos generales, la navegación privada implica que el navegador:

- Elimina las cookies tras cerrar la sesión.
- No se guarda ningún tipo de historial o formularios de auto-completado.
- No se guardan las contraseñas.
- Se borra la caché automáticamente al salir.

De igual modo, es preciso recordar lo que no hace:

- No proporciona conexiones seguras o cifradas.
- No oculta tu dirección IP.
- No evita que las páginas de Internet almacenen información sobre ti.
- No impide que tu navegación sea supervisada por el administrador de la red.
- No supone un anonimato total (aplicaciones de terceros como Flash pueden guardar sus propias cookies, etcétera).

3.1.5.3 Navegación anónima: TOR, I2P y proxies

Como se ha comentado, aún cuando naveguemos en modo privado, seguimos siendo identificables en la red: nuestra IP es visible y a partir de ahí se puede obtener nuestra posición geográfica aproximada, nuestro proveedor de servicio o incluso el nombre de la empresa en que trabajamos (si disponemos de una IP institucional).

La única forma de lograr un anonimato casi completo cuando navegamos es usar una conexión segura a una máquina denominada servidor http proxy. Un servidor http proxy es un ordenador que funciona como una pasarela a través de la cual se filtran nuestras peticiones de navegación por la web.

En la Figura 14 se muestra la función de intermediario que realiza el servidor proxy y como oculta la dirección IP a los servicios que se usen de la Web.



Figura 14. Navegación anónima

De esta forma, si queremos conectarnos a una página web, primero haremos la petición al servidor proxy, y será esta máquina a su vez la que haga la petición de carga a la página a la que queramos conectarnos, quedando nuestro ordenador oculto a ojos del servidor de esa página web, pues la IP que le consta a ese servidor web es la del proxy.

Cuando usamos este mecanismo varias veces seguidas de forma encadenada a través de varios servidores proxy, es casi imposible rastrear la IP original de nuestro ordenador, con lo que nuestra navegación es totalmente anónima.

TOR es el sistema de navegación anónima más popular. Se trata de una red gratuita gracias a la cual se puede navegar, chatear o descargar archivos de forma totalmente anónima. Pero TOR también es un conjunto de programas que posibilita el acceso a esta red.

Para usar TOR, debemos descargar la aplicación e instalarla. Existen también complementos para Firefox que facilitan la navegación anónima con TOR, así como versiones portables de la propia aplicación (Portable TOR), o basadas en el navegador Opera (Opera-TOR).

Además de TOR, existe una red de servidores proxy no cifrados y túneles VPN anónimos a través de los cuales poder hacer nuestra conexión, pero la fiabilidad es variable.

I2P y Freenet son redes P2P privadas, que sirven a comunidades anónimas a través de los cuales se intercambian grandes volúmenes de datos. La red I2P es una red dentro de Internet, de tal forma que sus comunicaciones son invisibles para el resto de usuarios de Internet.

3.1.6 Principales Navegadores

El mundo de los navegadores es muy variado y las cuotas de uso están muy repartidas incluso entre las distintas versiones de un mismo navegador como se puede ver en la siguiente Figura 15:

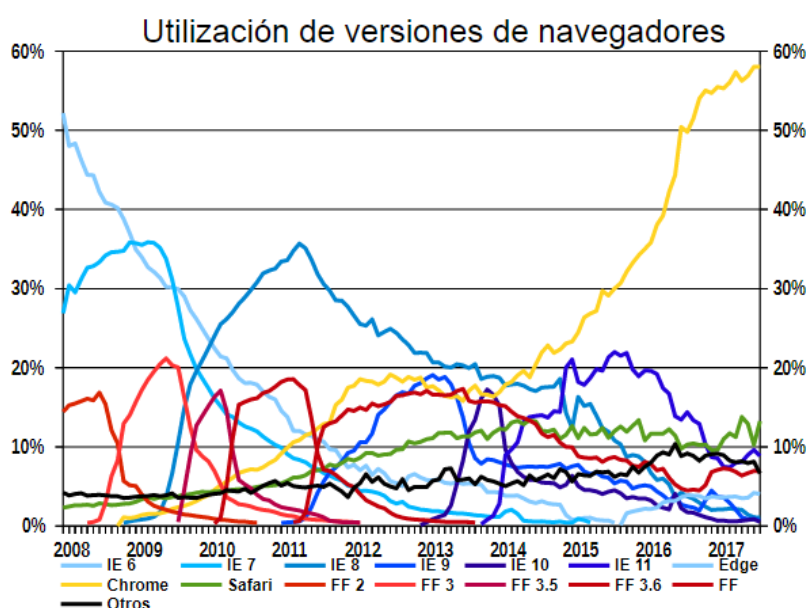


Figura 15. Principales navegadores

Abstrayéndonos de las versiones, los principales navegadores con mayor peso y a los que hay que prestar atención a día de hoy son los que se analizan en los siguientes apartados.

Google Chrome

Chrome es el nombre del navegador web desarrollado por Google y que salió a la luz en septiembre del 2008. Está disponible gratuitamente y cuenta con más de 750 millones de usuarios.

En la actualidad es líder del mercado, y tiene versiones disponibles para multitud de plataformas: windows, OS X, GNU/Linux (chromium), Android e iOS.

Internet Explorer

El antaño todopoderoso navegador de Microsoft, ha caído a cerca de un cuarto de cuota de mercado. Lastrado por sus problemas de seguridad, rendimiento e incumplimiento de estándares, mantiene una buena posición por la extensa base de usuarios de windows.

Microsoft Edge

Es el nuevo navegador de Microsoft que busca sustituir a Internet Explorer y que viene por defecto en con Windows 10. Este navegador se ha construido de cero sin depender de Internet Explorer para ser un navegador ligero, rápido y potente. Usa el motor de renderizado EdgeHTML que es una bifurcación del motor Trident usado por Internet Explorer.

Este nuevo navegador está en proceso de adaptarse a los nuevos estándares y a algunas tecnologías como ActiveX, y se considera que aun tiene que evolucionar como producto.

Firefox

Mozilla Firefox (llamado simplemente Firefox) es un navegador web libre y de código abierto desarrollado para Linux, Android, IOS, OS X y Microsoft Windows coordinado por la Corporación Mozilla y la Fundación Mozilla. Usa el motor Gecko para renderizar páginas web, el cual implementa actuales y futuros estándares web.

Safari

Safari es el navegador propietario desarrollado por Apple inicialmente para OS X (es su navegador por defecto), iOS (el sistema operativo móvil de Apple, presente en los iPhones, iPads e iPod Touch). Para windows dispone de una versión para lo que no se ofrece soporte desde el 2012.

Opera

Opera no solo es un navegador web, sino que es toda una suite (conjunto de aplicaciones) de internet que integra funcionalidades como la del propio navegador, un gestor de correo electrónico, gestor de contactos, lector RSS (sindicación de contenidos), cliente para charlas IRC e incluso un cliente BitTorrent. Pese a su posición marginal en el mercado, Opera fue pionero en la incorporación de características posteriormente adoptadas por todos los navegadores modernos, como las pestañas o la búsqueda integrada.

Posee funcionalidades muy avanzadas en seguridad, como la protección integrada contra phishing (robo de información confidencial) y malware, y el borrado sencillo de datos privados.

Una de las características definitorias de Opera ha sido, desde sus inicios, la variedad de sistemas operativos y plataformas en las que funciona: MS Windows, GNU/Linux, Mac OS X, FreeBSD, Windows Mobile, Blackberry OS, Maemo, Symbian, Android, iOS y otros sistemas operativos móviles con soporte de Java ME (Java Micro Edition, para dispositivos con recursos restringidos).

3.1.7 Comparativa de cuota de mercado de navegadores

El gráfico de la Figura 16 muestran los porcentajes de utilización de las diferentes versiones de Internet Explorer y Firefox desde finales de 2007 hasta la actualidad (basado en datos de la web Net Applications) como áreas apiladas, donde se puede ver la evolución en el uso de los navegadores y la progresión en la salida y adopción de las nuevas versiones de navegadores o incluso como han ido conviviendo estas versiones:

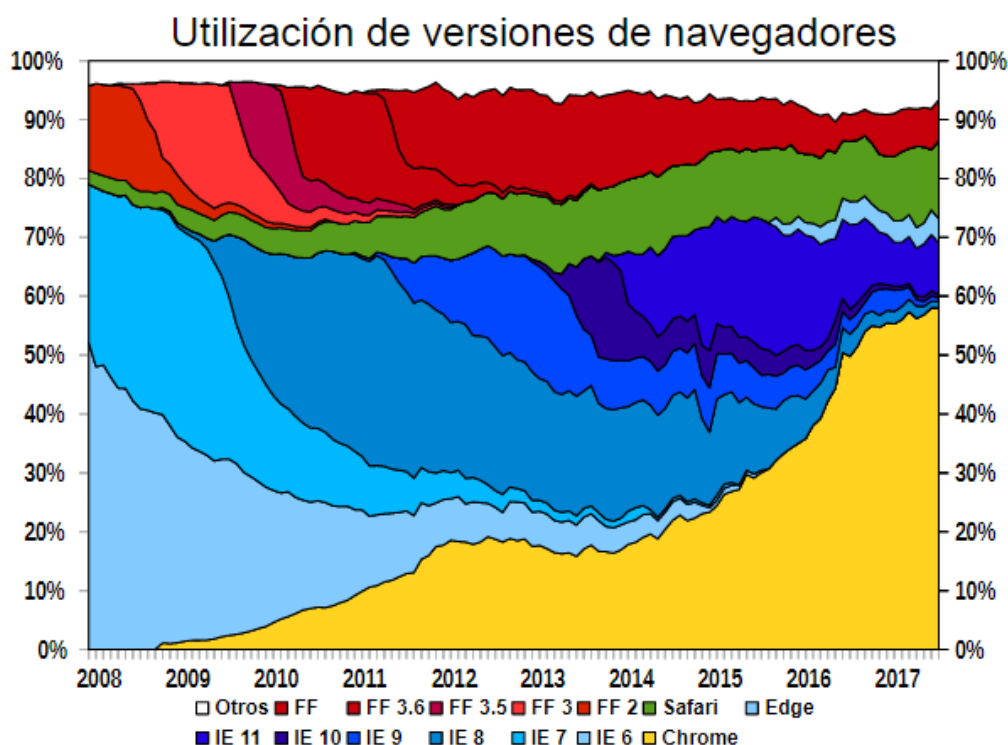


Figura 16. Estadísticas de uso 2007-2017

El mercado de los navegadores se ha ido fragmentando debido a que cada navegador saca nuevas versiones cada vez más a menudo (tanto para PC de escritorio como para móviles y tabletas) y las versiones antiguas se resisten a desaparecer. A mediados de 2017, podemos distinguir:

- Un navegador dominante, Google Chrome, que se acerca al 60% del mercado.
- Dos navegadores secundarios (IE11 y Safari), que se reparten el 20% del mercado.
- Dos navegadores minoritarios (Edge y Firefox), que se reparten poco más del 10% del mercado.
 - Microsoft Edge no está consiguiendo sustituir a Internet Explorer, entre otros motivos porque sólo está disponible en Windows 10.
 - Firefox está reduciendo su uso desde 2010.
- Tres navegadores en vías de desaparición (IE8, IE9 e IE10).
 - Internet Explorer 8 sigue utilizándose porque en Windows XP no se pueden instalar versiones posteriores de IE, pero está desapareciendo a la vez que ese sistema operativo.
 - Internet Explorer 9, al no estar disponible para Windows XP, creció más lentamente que las versiones anteriores. Llegó a ser el segundo navegador más utilizado a finales de 2012, pero a partir de la publicación de IE10 en febrero de 2013 su uso ha ido reduciéndose.
 - Internet Explorer 10 se publicó inicialmente sólo para Windows 8, por lo que su implantación fue mucho más lenta que la de versiones anteriores. A partir de febrero de 2013 estuvo disponible para Windows 7, lo que aceleró su implantación. Llegó a ser el segundo navegador más utilizado a mediados de 2013, pero a partir de la publicación de IE11 en octubre de 2013 su uso ha ido reduciéndose y prácticamente ha desaparecido.
- Dos navegadores ya desaparecidos (IE6 y IE7):
 - Internet Explorer 6 fue el navegador hegemónico entre 2002 y 2009. En 2012 era ya irrelevante, aunque seguía usándose en China. Desde mediados de 2015 se puede considerar desaparecido.

- Internet Explorer 7 fue durante unos meses en 2009 el navegador más utilizado. A partir de la publicación de IE8 en marzo de 2009 su uso se redujo rápidamente. A finales de 2012 era ya irrelevante. Desde principios de 2015 se puede considerar desaparecido.

En cualquier caso, el crecimiento de Google Chrome parece imparable y si se mantiene esta tendencia la única duda es saber qué porcentaje del mercado acabará acaparando.

3.2 Compatibilidad con estándares

3.2.1 Estándares Web

Los estándares web son un conjunto de recomendaciones emitidas por el World Wide Web Consortium (W3C) y otras organizaciones internacionales acerca de cómo crear e interpretar documentos basados en la web.

Estándares web es un término muy general utilizado para referirse a estándares y otras especificaciones técnicas que definen y describen aspectos de la World Wide Web. En años recientes, el término ha sido frecuentemente asociado con la tendencia de aprobar un conjunto de mejores prácticas estandarizadas para construir sitios web y a la filosofía del diseño y desarrollo web que incluye esos métodos.

Su objetivo es que los sitios web sean accesibles a más personas (incluidas aquellas con algún tipo de discapacidad), en más lenguajes y que se interpreten correctamente en cualquier dispositivo de acceso a internet.

En esencia, integran a un conjunto de tecnologías orientadas a beneficiar al mayor número posible de usuarios, asegurando la corrección, vigencia y perdurabilidad de todo documento web.

Los sitios que cumplen con los estándares web:

- Son más accesibles, permitiendo a personas con discapacidad acceder a su contenido.
- Son compatibles con todos los navegadores actuales, y lo serán con sus versiones futuras.
- Se visualizan correctamente en cualquier dispositivo de acceso: PCs, tablets o smartphones
- Son más fáciles de mantener y actualizar.
- Tienen una codificación más simple.

Los estándares web persiguen que el contenido de los sitios se visualice de la misma manera independientemente del navegador web con que accedamos al sitio, e incluso del dispositivo.

En la Figura 17 se muestran los ámbitos en los que se mueven los estándares web.



Figura 17. Ámbitos de los Estándares Web

Muchos estándares y especificaciones son interdependientes, algunos de los cuales gobiernan aspectos de la Internet, no sólo de la World Wide Web, que afectan de manera directa o indirecta el

desarrollo y la administración de sitios y servicios web. Algunos aspectos a considerar son la interoperabilidad, la accesibilidad y la usabilidad de páginas y sitios web.

De manera muy general, los estándares web, abarcan lo siguiente:

- Recomendaciones publicadas por el World Wide Web Consortium (W3C).
- Estándares de Internet (STD) documentados y publicados por Internet Engineering Task Force (IETF).
- Request For Comments —petición de comentarios (RFC)— , cuyos documentos son publicados también por la Internet Engineering Task Force.
- Estándares publicados por la Organización Internacional para la Estandarización (ISO).
- Estándares publicados por Ecma International.
- El estándar Unicode y otros varios reportes técnicos de Unicode (UTRs) publicados por el Consorcio Unicode.
- Nombres y números de registro mantenidos por la Internet Assigned Numbers Authority (IANA).

Cuando se discute sobre el uso de estándares, las siguientes publicaciones generalmente son vistas como fundamentales:

- Recomendaciones para lenguajes de marcado, como el lenguaje de marcas de hipertexto (HTML), lenguaje extensible de marcado de hipertexto (XHTML), Scalable Vector Graphics (SVG), y XForms, de W3C.
- Recomendaciones para hojas de estilo, especialmente hojas de estilo en cascada (CSS), de W3C.
- Estándares para ECMAScript, más comúnmente JavaScript, de Ecma International.
- Recomendaciones para Document Object Models (DOM), de W3C.
- Nombres y direcciones de página correctamente formados y demás recursos referenciados de sus (URIs), basado en RFC 2396, de IETF.
- El uso apropiado de los protocolos HTTP y MIME para desplegar la página, regresar datos pedir otros recursos referenciados a ésta, basado en RFC 2616, de IETF.

Normalmente, la accesibilidad web se basa en las denominadas Guías de Accesibilidad al Contenido Web publicados bajo la Iniciativa para la Accesibilidad Web de W3C, que se tratan más detalladamente en su tema correspondiente del temario con sus normas correspondientes.

El trabajo de la organización W3C hacia una web semántica está actualmente enfocado por publicaciones relacionadas al Marco de Descripción de Recursos (RDF), Gleaning Resource Descriptions from Dialects of Languages (GRDDL) y Web Ontology Language (OWL).

3.2.2 Web Standards Project (WaSP)

El Proyecto de Estándares Web (Web Standards Project, abreviado WaSP) es un grupo de desarrolladores web profesionales dedicado a fortalecer el uso de los estándares web recomendados por el World Wide Web Consortium, junto con otros grupos y otras entidades normativas.

Fundado en 1998, el Proyecto de Estándares Web promueve el uso de estándares que reducen el costo y la complejidad de desarrollo y que incrementan la accesibilidad y viabilidad de cualquier documento publicado en la Web. WaSP trabaja con compañías de navegadores y editores de páginas web para fortalecer el uso de estos estándares, porque estos están diseñados para entregar grandes beneficios al mayor número de usuarios web.

Actividades del proyecto:

- Desarrollo de la prueba Acid2 permite a los navegadores y otros motores de renderizado demostrar si satisfacen las especificaciones de CSS 1 y 2.
- Desarrollo de la prueba Acid3 permite a los navegadores demostrar si satisfacen las especificaciones de CSS 2.1, DOM y EcmaScript.

3.2.3 Cumplimiento de Estándares

3.2.3.1.1 Acid1

Acid1, originalmente llamado Box Acid Test, es una página de prueba para los navegadores web. Se desarrolló en octubre de 1998 y fue importante para establecer una base de referencia para la interoperabilidad de los primeros navegadores web, especialmente para las especificaciones Cascading Style Sheets 1.0.

Al igual que con las pruebas de ácido de oro, que producen una rápida y evidente de evaluación de la calidad de una pieza de metal, la web de las pruebas de ácido fue diseñada para producir una indicación clara del cumplimiento de las normas de un navegador web.

Los tests de Acid1 ponen a prueba muchas características en una sola página a fin de ser comparada contra una imagen de referencia. Actualmente casi todos los navegadores pasan la prueba.

Acid1 ha servido de ejemplo para Acid2 y Acid3.

3.2.3.1.2 Acid2

Acid2 es una página de prueba publicada y promovida por el Web Standards Project para detectar fallos en motores de renderizado usados por los navegadores para mostrar páginas web. Fue desarrollada en el espíritu de Acid1, una prueba relativamente estrecho de conformidad con el estándar Cascading Style Sheets 1.0 (CSS1), y fue lanzada el 13 de abril de 2005. Como con Acid1, una aplicación supera la prueba si el objeto en que se muestra la página de prueba coincide con una imagen de referencia.

Acid2 prueba aspectos de marcas HTML, estilo CSS, imágenes PNG, y data URIs. La página de prueba de Acid2 se muestra correctamente en cualquier aplicación que cumple con las especificaciones del World Wide Web Consortium y Internet Engineering Task Force que pertenecen a estas tecnologías.

Si el navegador cumple correctamente los estándares HTML y CSS 2.0, entonces se debe poder ver una cara sonriente. A pesar de lo simple del dibujo, cada línea está codificada de forma tal que dependa de la exacta implementación de varias exigencias que imponen los estándares.

Aunque en la época del lanzamiento de Acid2 ningún navegador pasaba la prueba, Acid2 fue diseñada con Microsoft Internet Explorer en mente. Los creadores de Acid2 lamentaban que Internet Explorer no cumplía con los estándares web y a causa de esto Internet Explorer mostraba las páginas web de forma diferente a otros navegadores. Cuando se encuentran tales discrepancias entre navegadores los desarrolladores web gastan tiempo en ajustar sus páginas web para que las páginas se muestren correctamente en navegadores diferentes. Acid2 representaba un desafío a Microsoft de poner Internet Explorer en línea con otros navegadores, haciendo más fácil diseñar páginas web que funcionen correctamente en cualquier navegador web.

En la Figura 18 se puede ver la imagen de referencia de Acid2. En la prueba real, la nariz se vuelve azul cuando el cursor está sobre ella.



Figura 18. Figura Test Acid2

El 31 de octubre de 2005, Safari fue el primer navegador que supera Acid2. Opera, Konqueror, Firefox, y otros siguieron. Con el lanzamiento de Internet Explorer 8 en 19 marzo de 2009, las

versiones más recientes de todos los navegadores web mayores ahora superan la prueba. Acid 2 fue sucedido por test Acid3.

3.2.3.1.3 Acid3

Acid3 es un sitio hecho por WaSP que pone a prueba los navegadores con los estándares web, especialmente los de DOM y ECMAScript. Desarrollado en abril de 2007 y fue lanzado el 3 de marzo de 2008. Acid2 se concentraba principalmente en CSS, en cambio Acid3 también se enfoca en tecnologías muy usadas en la Web 2.0, tales como ECMAScript y DOM Nivel 2. Algunas sub-pruebas también se encargan del lenguaje SVG, XML y del esquema data:URI. Sólo elementos de la especificaciones del 2004 son incluidos.

Cuando la prueba es satisfactoria, Acid3 muestra un porcentaje que gradualmente crece y además muestra rectángulos coloridos en el fondo. El porcentaje mostrado está basado en el número de sub-pruebas pasadas. Además, el navegador tiene que renderizar la página exactamente como la página de referencia es renderizada en este mismo navegador. A diferencia de la prueba Acid2, la referencia no es un archivo de mapa bits, permitiendo algunas diferencias en el renderizado de las fuentes.

En la Figura 19 vemos como es en la actualidad el resultado de test Acid3:

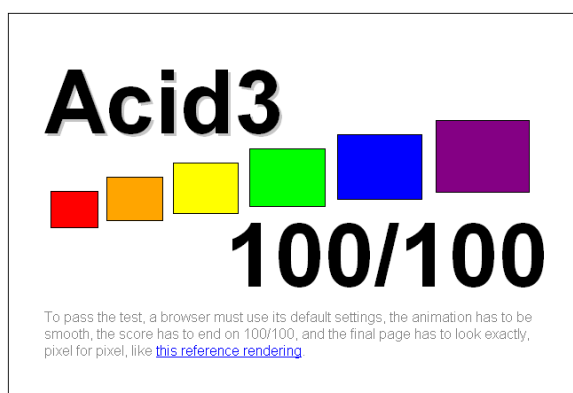


Figura 19. Figura Test Acid3

La parte principal de Acid3 está escrita en ECMAScript (JavaScript), y consiste de 100 subpruebas divididas en seis grupos, llamados "buckets", más cuatro pruebas especiales (0, 97, 98 y 99).

- **Bucket 1:** DOM Traversal, DOM Range, HTTP
- **Bucket 2:** DOM2 Core y DOM2 Events
- **Bucket 3:** DOM2 Views, DOM2 Style, Selectores de CSS3 y Media Queries
- **Bucket 4:** Comportamiento de tablas HTML y formularios cuando son manipulados por scripts y DOM2 HTML
- **Bucket 5:** Pruebas de la competición Acid3, (SVG, HTML, SMIL, Unicode...)
- **Bucket 6:** ECMAScript

El único requerimiento de la prueba es que el navegador tenga la configuración de forma predeterminada. El renderizado final debe mostrar un 100/100 y además no sólo debe ser idéntico al de referencia sino que además la animación debe ser fluida y no debe tardar más de 33 milisegundos.

3.2.3.1.4 Cumplimiento de los navegadores actuales

El cumplimiento de estándares es un aspecto complicado de medir ya que los tests suelen probar si un navegador soporta o no una determinada característica, no si lo hace correctamente y de acuerdo a las especificaciones.

Respecto al test Acid3 a día de hoy es superado sin problemas por los cuatro navegadores más destacados, por lo que se recomienda realizar test adicionales de CSS y HTML5 para aportar más información sobre el estado de madurez de los navegadores.

En la Figura 20 se compara el resultado obtenido por los tres principales navegadores en el test Acid3 y en otros dos test más como son HTML5Test y CSS3Test.

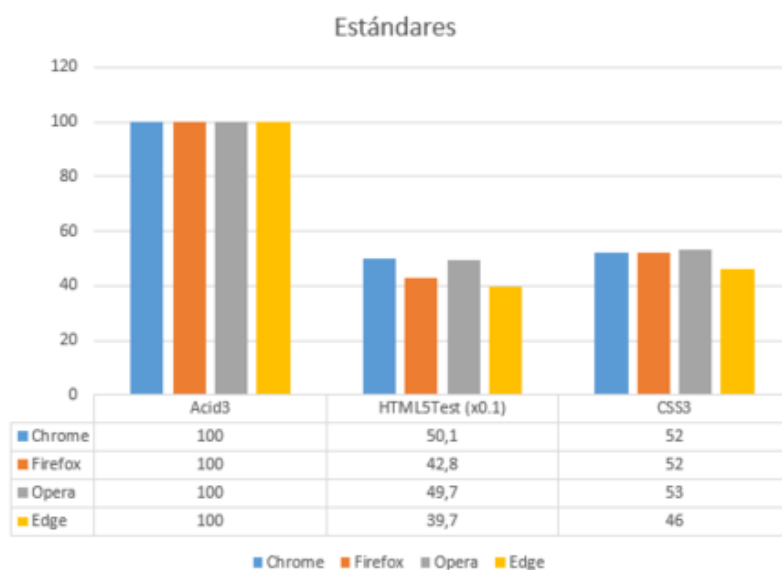


Figura 20. Comparativa de cumplimiento de estándares web

En las siguientes figuras, Figura 21, Figura 22 y Figura 23, se amplía las estadísticas de cumplimiento mostrando unas graficas que muestran la evolución de cumplimiento de HTML5 para dispositivos de escritorio, tablet y móviles obtenido de la web html5test.

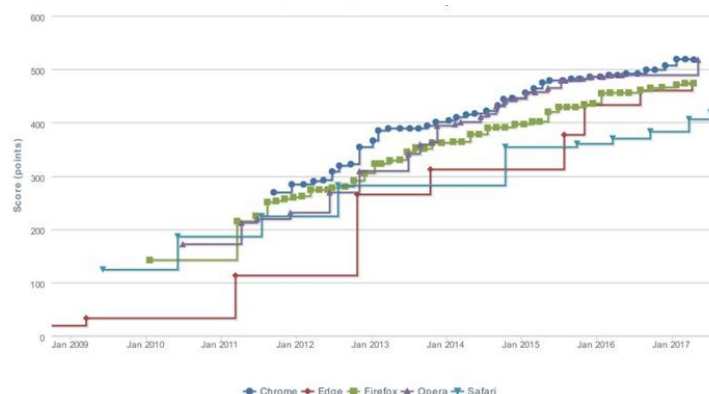


Figura 21. Evolución cumplimiento de estándares web navegadores de escritorio

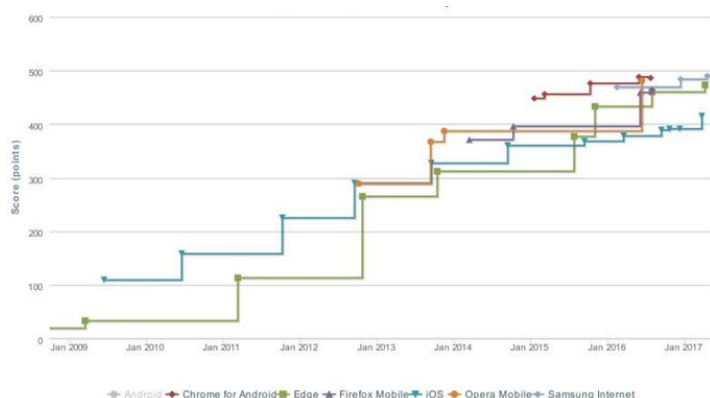


Figura 22. Evolución cumplimiento de estándares web navegadores de tablets

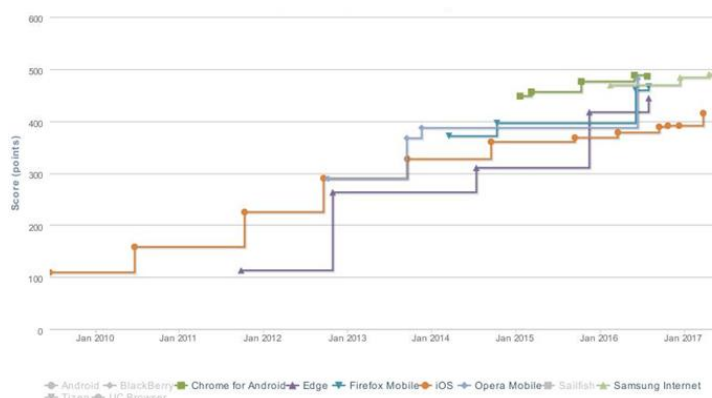


Figura 22. Evolución cumplimiento de estándares web navegadores de móvil

3.2.4 El Cross-browsing

Una aplicación web es considerada como cross-browser cuando se visualiza y funciona correctamente en todos los navegadores. El término cross-browsing se utiliza para denominar a aquellas soluciones o conocimientos específicos del desarrollo de aplicaciones cross-browser. Para que una solución sea cross-browsing hay que tener en cuenta que funcione correctamente tanto la parte visual como la parte funcional.

Un desarrollador web con conocimientos cross-browsing debe tener experiencia en desarrollo de aplicaciones soportadas en múltiples navegadores y también deberá tener conocimiento de los estándares de los principales lenguajes de programación front-end.

La diferencia entre una aplicación web cross-browser y una aplicación web multi-browser es que la primera se visualiza y funciona correctamente en todos los navegadores. Se diferencia de multi-browser, porque una aplicación es multi-browser cuando da soporte específico a varios navegadores, pero no a todos. Es decir, cross-browser es un término más amplio y que implica más esfuerzo de implementación.

En ocasiones es imposible conseguir que una misma solución funcione en todos los navegadores y es necesario desarrollar dos soluciones diferentes y aplicar cada una en una serie de navegadores diferentes.

La mayoría de sitios web son multi-browser, pero dan soporte a tantas versiones de navegadores diferentes que se aproximan mucho a ser cross-browser. Hay versiones de navegador tan antiguas y que utilizan tan pocos usuarios que no tiene sentido darles soporte por el esfuerzo que representan.

Para seleccionar los navegadores a los que dar soporte lo recomendable es realizar un estudio de qué navegadores son mayoritariamente usados por los visitantes del sitio web a adaptar para establecer la prioridad.

Actualmente, a mediados de 2017, se considera necesario dar soporte a los navegadores mayoritarios siguientes:

- Última versión estable de Firefox.
- Última versión estable de Chrome.
- Últimas dos versiones de Safari.
- Internet explorer 8, 9, 10, 11 y Edge.

El resto de versiones y el resto de navegadores son utilizados por un porcentaje muy bajo de usuarios. Por este motivo, suelen implementarse soluciones multi-browsing y se les suele llamar soluciones cross-browsing ya que, aunque sea incorrecto llamarlas así, en la práctica funcionan en todos los navegadores (en todos los navegadores a los que tiene sentido dar soporte).

Cuando se aborda un proyecto de creación de un nuevo sitio web se deben seguir todos los estándares que se han visto en este tema: HTML5, CSS3 y ECMAScript, de esta manera se simplifica la compatibilidad ya que los navegadores modernos también siguen en su mayoría estos estándares.

Los pasos a seguir en caso de detectar incompatibilidad con algún navegador deben ser:

1. Averiguar el motivo por el que no funciona. Es decir, quien no cumple el estándar la aplicación o el navegador.
2. Analizar la conveniencia de dar soporte a ese navegador o versión.
3. En caso de ser necesario busca una solución general que cubra el problema en todos los casos.
4. Solo en caso de no ser posible una solución general se debe implementar una solución particular para un navegador o versión específicos.

Es recomendable evitar siempre que sea posible las soluciones específicas de un navegador ya que dificultan enormemente el mantenimiento y la evolución de las soluciones web y puede traer problemas inesperados con los cambios de versión.

La regla que se debe aplicar es que el 90% de los problemas cross browsing pueden resolverse con una solución, menos del 9% necesitan dos soluciones y solamente casos muy extremos de menos del 1% necesitan más de dos soluciones.

No confundir este concepto con el Cross-site scripting o XSS, que es un tipo de inseguridad informática o agujero de seguridad típico de las aplicaciones Web, que permite a una tercera persona inyectar en páginas web visitadas por el usuario código JavaScript o en otro lenguaje similar (ej: VBScript), evitando medidas de control como la política del mismo origen.

4 Conclusiones

A lo largo del tema se han descrito los lenguajes y herramientas principales para la utilización de redes globales, centrándonos en HTML y XML. Por otro lado, se han analizado los navegadores web y los elementos de compatibilidad con estándares.

Ha transcurrido mucho tiempo (desde la perspectiva de la evolución de la tecnología) desde los inicios de HTML. Durante los más de 20 años de vida de este lenguaje se ha creado un ecosistema que ha ido evolucionando a la vez que lo hacían las tecnologías que permitían tecnológicamente dotar de mayores capacidades a este lenguaje y su entorno. Desde un inicio en el que las carencias se suplían a base de tecnologías no estándar desarrolladas alrededor de éste, hasta la situación actual que nos lleva hacia una estandarización del trío HTML/CSS/JavaScript como eje principal de la web actual.

De forma paralela y anterior a HTML se establecían otros lenguajes de marcado que influenciarían a HTML tal como SGML del cuál evolucionó XML. Si bien se han realizado esfuerzos por unificar de forma completa HTML y XML en XHTML, en la actualidad son tecnologías complementarias pero distinguidas la una de la otra. HTML se ha centrado en la presentación al usuario mientras que XML se utiliza en la actualidad más para la comunicación entre máquinas. La proliferación de XML ha permitido que el estándar se convierta también en un estándar de facto como lenguaje a la hora de comunicar máquinas y sistemas diversos.

Así pues ambas tecnologías son ampliamente utilizadas en la Administración Pública española. HTML se utiliza hoy en día como la tecnología base del instrumento principal para comunicarse con el ciudadano de forma telemática, mientras que XML se utiliza tanto interna como externamente en multitud de trámites telemáticos facilitando el uso de un lenguaje común.

Es imposible conocer de forma certera cuál será el futuro de estas tecnologías, pero se puede afirmar que ambas son de vital importancia en la administración digital.

La web vivió unos primeros años de desarrollo frenético gracias a la competencia entre Netscape e Internet Explorer. Los años de dominio absoluto de Internet Explorer fueron años de estancamiento de la web (desde el punto de vista técnico, no en cuanto a cantidad de contenidos). Afortunadamente, ese predominio se empezó a erosionar en 2004 con Firefox y desde 2009 hasta 2014, la competencia entre Firefox, Chrome e Internet Explorer provocó grandes mejoras en todos los navegadores e innovaciones técnicas que ya disfrutamos. Desde 2014, Google Chrome se ha convertido en el navegador dominante y no parece haber alcanzado todavía su techo.

El predominio de Windows daba a Internet Explorer una ventaja fundamental, ya que Internet Explorer viene incluido de serie en Windows y cada ordenador que se vende es un ordenador con Internet Explorer, salvo que el usuario tenga la voluntad y conocimientos para cambiar a otro navegador. El crecimiento de los dispositivos móviles ha cambiado el terreno de juego. La plataforma mayoritaria en los dispositivos móviles es Android y su navegador nativo, Chrome, cuenta con ventaja.

Esperemos que en los próximos años el predominio de Chrome no signifique un estancamiento del mercado y que la competencia de Firefox, Internet Explorer y Safari obligue a Google a seguir mejorando y a respetar las recomendaciones del W3C. Todos nosotros saldremos ganando si la web sigue siendo ese campo de juego neutral que soñó Berners Lee hace 25 años.

Orientando el tema al examen tipo test, cabe destacar que es un tema del cual pueden aparecer muy diferentes preguntas, al incluir tecnologías muy extendidas y utilizadas en la administración y que se encuentran en un continuo cambio. Entre ellas se podrían destacar las relacionadas con lo siguiente: relacionadas con conocer las versiones y nombres de tecnologías; relacionadas con qué características y funcionalidad tiene una determinada tecnología o versión; novedades relacionadas sobre estas tecnologías, por tanto es recomendable tener presente la última actualización del tema; detalles técnicos; preguntas relacionadas con la legislación aplicable en la administración y estas tecnologías...

Además se recomienda revisar las preguntas tipo test de ejemplo incluidas en el tema, pues que incluyen algunas preguntas del estilo a las preguntadas en exámenes de años anteriores así como otras que se cree podrían encajar con lo que se suele preguntar. Esto nos permite hacernos una idea del tipo de preguntas que suelen preguntarse, que junto con el Resumen Esquemático nos ayuda a comprender qué es lo más importante que debe memorizarse de cara al examen tipo test.

GLOSARIO

AJAX - Asynchronous JavaScript and XML
API - Application Program Interface
ASP - Active Server Pages
CGI - Common Gateway Interface
CSS - Cascading Style Sheets
DOM - Document Object Model
DTD – Document Type Definition
HTML - HyperText Markup Language
HTTP - Hypertext Transfer Protocol
IETF - Internet Engineering Task Force
JSON - JavaScript Object Notation
JSP - JavaServer Pages
PHP - Hypertext Preprocessor
RSS - Really Simple Syndication
SAX - Simple API for XML
SEO - Search engine optimization
SOAP - Simple Object Access Protocol
UDDI - Universal Description, Discovery and Integration
URL - Uniform Resource Locator
W3C - World Wide Web Consortium
WHATWG - Web Hypertext Application Technology Working Group
WSDL - Web Services Description Language
XBRL- eXtensible Business Reporting Language
XHTML - Extensible Hypertext Markup Language
XML - Extensible Markup Language
XPath - XML Path Language
XSL - EXtensible Stylesheet Language
XSLT - XSL Transformations

RESUMEN ESQUEMÁTICO

HTML

- La Web, entendida como servicio o aplicación de Internet sigue la **arquitectura cliente – servidor**.
- El **protocolo** de comunicaciones mediante el que dialogan el cliente (navegadores) y servidores web es **HTTP** (Hypertext Transfer Protocol) **transmitiendo** principalmente **HTML** junto a otras tecnologías relacionadas como JavaScript, CSS, JSON o imágenes.
- **HTTP es stateless**, usándose otras tecnologías como las cookies para simular sesiones.
- La **versión segura de HTTP es HTTPS**, y utiliza cifrado basado en SSL/TLS para crear un canal cifrado entre el servidor y el cliente.
- En mayo de 2015 se publicó el RFC 7540 con la especificación de **HTTP/2**, la cual pretende ofrecer mayor velocidad, reducir el uso de recursos en la red y el servidor, mejorar el rendimiento de la red y la comprensión de cabeceras.
- Para especificar la ubicación de los documentos en la web se utilizan las URLs y URIs.
- HTML es un **lenguaje de marcas** o etiquetas, y permite describir el contenido y formato de los documentos web, siendo su **versión actual HTML5**.
- **DOM** es una API que proporciona un conjunto estándar de objetos para representar documentos HTML y XML, y un modelo estándar sobre cómo pueden combinarse dichos objetos. El DOM representa los objetos de un documento HTML o XML organizados jerárquicamente.
- Las hojas **CSS** (Cascading Style Sheets) permiten modificar el **aspecto de una página web**, aunque son dependientes de HTML. Su versión actual es CSS3.
- **JavaScript** es un lenguaje de programación muy relacionado con HTML que permite interactuar con el Modelo de Objetos del Documento (DOM). Los programas de JavaScript se incrustan en las páginas HTML, o se vinculan a ellas, y se ejecutan en el navegador web con el objeto de dotar a las páginas web de mayor interactividad con el usuario.
- El código HTML puede ser generado de forma estática o dinámica. La generación dinámica se puede realizar en el servidor (utilizando tecnologías como PHP o Servlets) o en el cliente (utilizando tecnologías como JavaScript).
- Las tecnologías web son ampliamente utilizadas en la administración. La **Norma Técnica de Interoperabilidad** (NTI) de Catalogo de estándares establece un catálogo formado por un conjunto mínimo de estándares que satisfacen lo previsto en el artículo 11 del Real Decreto 4/2010, de 8 de enero, y que dan soporte al resto de Normas Técnicas de Interoperabilidad. En él se encuentran numerosas tecnologías web, incluyendo HTML, HTTP y URL/URI.

XML

- **XML** (eXtensible Markup Language) es un lenguaje de marcas que permite describir información o **contenido de forma completamente separada de su presentación o formato**. XML está estandarizado por el W3C (World Wide Web Consortium), siendo su versión actual XML 1.1.
- En XML las etiquetas y atributos a utilizar son ilimitados. XML permite definir otros lenguajes o gramáticas.
- XML aporta dos herramientas fundamentales para definir lenguajes basados en XML y, una vez definidos esos lenguajes, validar documentos XML contra su definición. Estas herramientas son los **DTDs** (Document Type Definitions) y los **XML Schemas**.
- Cuando un documento respeta las reglas básicas de sintaxis XML (abrir y cerrar las etiquetas, etc.) se dice que está **bien formado**.

- Cuando un documento XML bien formado es conforme a un DTD o un XML Schema, se dice que el documento es **válido**.
- Los dos modelos de funcionamiento principales de procesamiento de documentos XML son el modelo **DOM** (Document Object Model) y el modelo **SAX** (Simple API for XML). El modelo DOM mantiene la representación en memoria, el SAX procesa el documento poco a poco.
- **XSL**, que es un estándar del W3C, permite complementar la información de un documento XML con una hoja de estilos (instrucciones de formateo). Por tanto, XSL es a XML lo que CSS es a HTML.
- Los estándares de XSL más destacados son: **XSLT (XSL Transformations)** para transformar un documento XML original en otro documento que resulte de la transformación del primero; **XPath (XML Path Language)** para referenciar partes específicas de un documento XML; y **XSL-FO (XSL Formatting Objects)** para la presentación de un documento XML.
- Los lenguajes de intercambio de información de los Servicios Web SOAP, WSDL y UDDI están basados en XML.
- XBRL es un nuevo lenguaje basado en XML que permite que el intercambio de información contable, financiera y tributaria entre empresas, organismos y la propia Administración Pública.
- Tanto HTML como XML son ampliamente utilizadas en la Administración. Su uso está respaldado por el Esquema Nacional de Interoperabilidad (ENI) y las Normas Técnicas de Interoperabilidad (NTIs).

Navegadores web

1. Hitos Historia

- El primer navegador web fue desarrollado por Tim Berners-Lee, en la CERN, a finales de 1990 y principios de 1991;
- Enero de 2003, Apple lanzó al mercado el navegador web Safari.
- A finales de 2004 aparece en el mercado Mozilla Firefox,
- Septiembre de 2008, Google Chrome aparece en el mercado.
- En diciembre de 2011, Chrome superó a Internet Explorer 8.0 como el navegador más utilizado a nivel mundial.

2. Conceptos

- Un **navegador web** es una aplicación que permite el acceso a internet, interpreta la información de archivos etiquetados en HTML y los presenta en pantalla según las directrices de presentación codificadas en una hoja de estilos CSS.
- Una **URL** es la cadena de caracteres con la que se asigna una dirección única a todos y cada uno de los recursos de información disponibles en internet.
- El **protocolo DNS** asigna nombres a direcciones IP (es lo que permite encontrar un servidor tecleando un nombre).
- Un **puerto** es una interfaz a través de la cual pueden enviarse y recibirse datos para comunicarse con otro programa o aplicación a través de una red telemática.

3. Ciclo de solicitud-respuesta y transacciones HTTP

1. Todo comienza cuando escribimos la URL en la barra de direcciones del navegador.
2. Un servidor de nombres DNS busca la página que hemos escrito, averigua su dirección IP y la devuelve al navegador.
3. El ordenador o dispositivo del usuario envía una solicitud al servidor identificado por la dirección IP y espera respuesta.
 - a. Si todo va bien, el servidor envía un breve mensaje de retorno al navegador indicando que todo está correcto (transacciones HTTP), seguido del contenido de la propia página web.

- b. Si hay problemas, el servidor envía al navegador un código de error HTTP que identifica la naturaleza del problema (el más conocido es el error 404: "no se encuentra la página")

4. Componentes de un navegador

- a. **Interfaz Gráfica de Usuario**
- b. **Motor de Renderizado**
 - Webkit
 - Gecko
 - Trident y Tasman
 - Blink
 - Presto
- c. **Motores JavaScript**
- d. **Complementos**

5. Tipos de Navegación

- **Navegación segura:** El protocolo HTTPS (Hypertext Transfer Protocol Secure, o protocolo HTTP seguro) garantiza que las sesiones de navegación están cifradas, por lo que la transferencia de datos es segura.
- **Navegación privada:** Con la navegación privada, el navegador no deja en el ordenador ningún rastro de las páginas que visita (cookies, caché e historial).
- **Navegación anónima:** TOR, I2P y proxies gratuitos

6. Principales Navegadores

- Google Chrome
- Internet Explorer
- Microsoft Edge
- Firefox
- Safari
- Opera

Compatibilidad con estándares

1. Los **estándares web** son un conjunto de recomendaciones emitidas por el World Wide Web Consortium (W3C) y otras organizaciones internacionales acerca de cómo crear e interpretar documentos basados en la web.

2. Publicaciones fundamentales:

- **Recomendaciones para lenguajes de marcado**, como el lenguaje de marcas de hipertexto (HTML), lenguaje extensible de marcado de hipertexto (XHTML), Scalable Vector Graphics (SVG), y XForms, de W3C.
- **Recomendaciones para hojas de estilo**, especialmente hojas de estilo en cascada (CSS), de W3C.
- **Estándares para ECMAScript**, más comúnmente JavaScript, de Ecma International.
- **Recomendaciones para Document Object Models (DOM)**, de W3C.
- **Nombres y direcciones de página** correctamente formados y demás recursos referenciados de sus (URIs), basado en RFC 2396, de IETF8
- El uso apropiado de **los protocolos HTTP y MIME** para desplegar la página, regresar datos pedir otros recursos referenciados a ésta, basado en RFC 2616, de IETF9

3. Web Standards Project (WaSP)

- Es un grupo de desarrolladores web profesionales dedicado a fortalecer el uso de los estándares web recomendados por el World Wide Web Consortium
- Actividades:
 - La prueba Acid2 permite a los navegadores y otros motores de renderizado demostrar si satisfacen las especificaciones de CSS 1 y 2.

- La prueba Acid3 permite a los navegadores demostrar si satisfacen las especificaciones de CSS 2.1, DOM y EcmaScript.
 - La parte principal de Acid3 está escrita en ECMAScript (JavaScript), y consiste de 100 sub-pruebas divididas en seis grupos, llamados "buckets", más cuatro pruebas
 - **Bucket 1:** DOM Traversal, DOM Range, HTTP
 - **Bucket 2:** DOM2 Core y DOM2 Events
 - **Bucket 3:** DOM2 Views, DOM2 Style, Selectores de CSS3 y Media Queries
 - **Bucket 4:** Comportamiento de tablas HTML y formularios cuando son manipulados por scripts y DOM2 HTML
 - **Bucket 5:** Pruebas de la competición Acid3, (SVG, HTML, SMIL, Unicode...)
 - **Bucket 6:** ECMAScript
4. **Cross-browsing:** Se dice que una aplicación web es cross-browser cuando se visualiza y funciona correctamente en todos los navegadores.

PREGUNTAS DE TEST

- 1) XML y HTML5 se caracterizan porque ambos
 - a) se apoyan en JavaScript.
 - b) se basan en el protocolo HTTP.
 - c) son lenguajes de marcado.
 - d) son conformes a SOAP.
- 2) AJAX es:
 - a) Un lenguaje de programación basado en la codificación de programas en XML.
 - b) Una filosofía o técnica de desarrollo de aplicaciones web que se apoya, entre otras tecnologías, en XML y JavaScript.
 - c) Un servidor de aplicaciones Internet desarrollado en Java cuyas páginas web están escritas en XHTML.
 - d) Una alternativa a SOAP en la descripción de web services.
- 3) Respecto a HTML5, señale la respuesta correcta:
 - a) Su uso está recomendado para el desarrollo web de los sitios de la Administración General del Estado, siempre que no se incumpla la normativa de accesibilidad.
 - b) En ningún caso se recomienda el uso de HTML5 en el desarrollo web de los sitios de la Administración General del Estado.
 - c) Simplifica el desarrollo de sitios web porque elimina el uso de CSS (Cascading Style Sheets).
 - d) No permite incluir vídeo de forma nativa, siendo necesario recurrir siempre a complementos (plugins) externos.
- 4) Indique quién desarrolló el lenguaje JavaScript
 - a) Microsoft
 - b) SUN
 - c) Netscape
 - d) W3C
- 5) ¿Qué plugin se necesita para poder ejecutar HTML5?
 - a) Adobe Flash.
 - b) W3C Standard Plugin.
 - c) Internet Explorer.
 - d) Ninguno, sólo tener un navegador con soporte nativo.
- 6) Gracias a CSS3:
 - a) Existe una separación total entre contenido y presentación en HTML/CSS3.
 - b) Podemos grabar vídeos utilizando la webcam del ordenador sin necesidad de plugins.
 - c) Se permite el uso de animaciones sin necesidad de plugins adicionales.
 - d) Permite rotar elementos 2D pero no 3D.
- 7) La generación dinámica de HTML en el servidor
 - a) supone una sobrecarga con respecto a una página estática
 - b) permite adaptar el contenido al usuario
 - c) no es de uso obligado para server contenido web al cliente
 - d) todas las anteriores son ciertas
- 8) ¿Qué afirmación es INCORRECTA?
 - a) XHTML no es totalmente compatible hacia atrás.
 - b) Una página web puede contener varios ficheros CSS.
 - c) Adobe Flash no es parte del estándar de CSS.
 - d) En HTML5 toda página web debe incluir CSS para formatearla.
- 9) En relación con lenguajes de marcado, indique cuál de las siguientes afirmaciones es INCORRECTA.
 - a) XSLT es un lenguaje de marcado para las transformaciones de documentos XML.
 - b) El XML describe el contenido de la información y la estructura, mientras que HTML está orientado a la presentación de la información.
 - c) XPath permite buscar y seleccionar parte de un fichero XML sin utilizar su estructura.
 - d) CSS permite dar formato tanto a ficheros HTML como a ficheros XML.
- 10) ¿Cómo se puede validar documentos en XML?
 - a) Usando XSL Schema.
 - b) Usando XSLT.
 - c) Usando un Applet de validación.
 - d) Usando un DTD.
- 11) HTML está estandarizado por:
 - a) World Wide Web Consortium - W3C.
 - b) Internet Engineering Task Force - IETF.
 - c) Institute of Management Accountants - IMA.
 - d) International Accounting Standard Board - IASB.

- 12) ¿En cuál de los siguientes componentes de XML se define la estructura de los datos que va a contener un formulario XML?
 - a) Manifiestos (XSF).
 - b) Vistas (XLS).
 - c) Esquemas (XSD).
 - d) Datos (XML).
- 13) En análisis de XML, indique cuál de las siguientes expresiones NO es correcta:
 - a) El modelo DOM procesa el documento XML completo y lo almacena en memoria.
 - b) En el modelo SAX puede procesar el documento XML a través de eventos.
 - c) El modelo SAX procesa el documento XML y genera un árbol.
 - d) El modelo DOM procesa el documento XML y lo modeliza en forma de árbol.
- 14) El estándar WSDL:
 - a) Es el protocolo para efectuar llamadas a métodos de objetos remotos mediante peticiones XML.
 - b) Describe la interfaz de servicio.
 - c) Es el registro público de servicios.
 - d) Es el protocolo de transporte.
- 15) Qué es Atom?
 - a) Sistema de análisis XML.
 - b) Tecnología de sindicación de contenidos.
 - c) Lenguaje que permite el intercambio de información financiera de las empresas.
 - d) Sintaxis de una DTD.
- 16) El XSLT transforma:
 - a) Sólo de XML a XML
 - b) De XML a otro documento
 - c) Sólo de XML a HTML
 - d) De un documento a XML
- 17) ¿Cuál de las siguientes afirmaciones sobre RSS es errónea?:
 - a) RSS es un formato para la sindicación de contenidos de páginas web.
 - b) RSS es un agregador de contenidos de diversas páginas webs de noticias
 - c) RSS es parte de la familia de los formatos XML
 - d) Es un acrónimo de "Really Simple Syndication"
- 18) ¿Cuál de las siguientes afirmaciones con respecto a los XML Schemas es incorrecta?:
 - a) Los XML Schemas utilizan sintaxis XML
 - b) Es imposible referenciar múltiples XML Schemas desde el mismo documento XML
 - c) Los XML Schemas soportan namespaces
 - d) Los XML Schemas permiten definir tipos de datos
- 19) Un documento XML bien formado:
 - a) siempre es válido.
 - b) nunca es válido.
 - c) es válido si además es conforme a una DTD o un XML Schema.
 - d) se considera como tal si es conforme a una DTD o un XML Schema.
- 20) El XML (Extensible Markup Language) ha sido desarrollado por el W3C. Indique cuál de las siguientes afirmaciones es falsa:
 - a) Dada su extensibilidad no se soporta en bases de datos sino que requiere formatos de texto plano
 - b) Proviene del SGML
 - c) Pretende ser un estándar para el intercambio de información entre plataformas no circunscrito exclusivamente a Internet
 - d) No es un lenguaje propiamente dicho sino un metalenguaje creado para definir lenguajes que cubran diferentes necesidades
- 21) ¿Cómo se llama el protocolo que usa el navegador para encontrar la IP que corresponde a un nombre de dominio?
 - a) TCP/IP
 - b) URI
 - c) DNS
 - d) URL
- 22) ¿Cuál de las siguientes no es una parte de un navegador actual?
 - a) La Interfaz de usuario
 - b) El motor de renderizado
 - c) El intérprete JavaScript
 - d) Todas son partes de un navegador.
- 23) ¿Cuál de los siguientes no es un motor de renderizado?
 - a) Webkit
 - b) PowerKit
 - c) Blink

- d) Trident y Tasman
- 24) ¿Qué afirmación es correcta?
 - a) El protocolo HTTPS garantiza que las sesiones de navegación están cifradas, por lo que la transferencia de datos es segura.
 - b) La navegación privada, el navegador no deja en el ordenador ningún rastro de las páginas que visita y es igual que la navegación segura con https.
 - c) Es recomendable usar navegación anónima en las comprar en la red TOR o algún proxy gratuito.
 - d) Es fundamental que te encuentres dentro de una sesión anónima cuando introduzcas o manejes datos sensibles, como datos bancarios, académicos o de compras.
- 25) ¿Cuáles de los siguientes no están entre los 3 navegadores más usados actualmente?
 - a) Opera
 - b) Safari
 - c) Chrome
 - d) Internet Explorer
- 26) ¿Cuál es el Test actual que permite ver el estado de compatibilidad de un navegador moderno?
 - a) W3C Test
 - b) Acid3 Test
 - c) ACDC Test
 - d) Bucket6 Test
- 27) ¿Qué consorcio se encarga de los estándares de Internet?
 - a) OASIS
 - b) ISO
 - c) W3C
 - d) IETF
- 28) Si quiero que mi página web sea compatible con la mayoría de los navegadores cuál de las siguientes prácticas no es útil
 - a) Cumplir con los estándares
 - b) Aplicar técnicas de cross-site scripting
 - c) Cumplir con las normas WAI de accesibilidad
 - d) Aplicar técnicas de cross-browsing

SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	c
2	b
3	a
4	c
5	d
6	c
7	d
8	d
9	c
10	d
11	a
12	c
13	c
14	b
15	b
16	b
17	b
18	b
19	c
20	a
21	c
22	d
23	b
24	a
25	a
26	b
27	c
28	b

BIBLIOGRAFÍA BÁSICA

1. CSS: <http://www.w3.org/Style/CSS/>
2. CSS: <http://www.w3.org/Style/CSS/current-work>
3. CSS: http://www.w3schools.com/css/css3_intro.asp
4. DTD: <http://www.w3.org/TR/2004/REC-xml11-20040204> (apartado 2.8 Prolog and Document Type Declaration)
5. HTML: <http://www.w3.org/html/>
6. HTML: http://www.w3schools.com/html/html5_intro.asp
7. HTML: <http://www.w3.org/TR/html5/>
8. HTTP: <https://www.ietf.org/rfc/rfc2616.txt>
9. XML: <http://www.w3.org/XML>
10. XML Schemas: <http://www.w3.org/XML/Schema>
11. Navegadores – Universidad de Alicante:
https://rua.ua.es/dspace/bitstream/10045/54704/3/ci2_basico_2015-16_Navegadores.pdf
12. Navegadores: https://es.wikipedia.org/wiki/Navegador_web
13. Estándares Web: https://es.wikipedia.org/wiki/Est%C3%A1ndares_web

BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

14. AJAX: <https://api.jquery.com/category/ajax/>
15. DOM: <http://www.w3.org/DOM>
16. DTD: <http://www.w3schools.com/dtd/>
17. dXSL: <http://www.w3.org/Style/XSL/>
18. Generación dinámica de HTML: <http://www.desarrolloweb.com/manuales>
19. HTML: <http://drafts.htmlwg.org/html/master/>
20. HTML: <http://www.w3schools.com/html/default.asp>
21. HTTP: <http://www.jmarshall.com/easy/http/>
22. HTTP: <https://http2.github.io/faq/>
23. HTTP: <http://tools.ietf.org/html/draft-ietf-httpbis-http2-04>
24. HTTP: <http://dev.chromium.org/spdy>
25. HTTP: <https://tools.ietf.org/html/rfc2818>
26. JavaScript: <http://www.w3schools.com/js/>
27. RSS: <http://www.rssboard.org/rss-specification>
28. SAX: <http://www.saxproject.org/>
29. Servicios Web XML: <http://www.w3.org/2002/ws/>
30. SOAP: <http://www.w3.org/TR/soap/>
31. UDDI: <http://uddi.xml.org/>
32. UDDI: <https://www.oasis-open.org/standards#uddiv3.0.2>
33. WSDL: <http://www.w3.org/TR/wsdl>

- 34. XBRL: <http://www.xbrl.es/>
- 35. XBRL: <https://www.xbrl.org/>
- 36. XHTML: <http://www.w3.org/TR/xhtml1/>
- 37. XML: <http://www.w3schools.com/xml/default.asp>
- 38. XPATH: <http://www.w3.org/TR/xpath>
- 39. XSLT: <http://www.w3.org/TR/xslt>